



Université de Sherbrooke

INF-808 — Réactions aux Attaques Informatiques

La Détection de Cryptojacking dans les environnements conteneurisés

Rapport de projet

Côté-Handfield, Léna	Matricule : cotl2716
Halimi, Amine	Matricule : hala4458
Hébert, Alphée	Matricule : heba1204

April 9, 2026

Contents

1	Problématique à résoudre	3
1.1	Le crypto-jacking comme menace informatique moderne	3
1.2	Le cryptomining malveillant dans les environnements conteneurisés	3
1.3	Réalité du phénomène	3
1.4	Conclusion	4
2	État de l'art	4
2.1	Problématique antérieure (et leur solution)	4
2.1.1	Cas de Uptycs:	4
2.2	Analyse des outils existants	5
2.2.1	Aqua Trivy	5
2.2.2	Docker Scout	5
2.2.3	Atouts de ces outils	6
2.2.4	Limites de ces outils	6
2.3	Indicateurs utiles	6
3	Retour sur la Problématique	7
4	Justification du projet et délimitation de son périmètre	8
5	Définition de l'implémentation	8
5.1	Squelette de l'architecture de l'outil	8
5.1.1	Principes généraux	12
5.1.2	Architecture globale proposée	12
5.2	Plan de projet	13
5.2.1	Phase 0: Cadrage expérimentale	13
5.2.2	Phase 1: Mise en place de l'environnement d'exécution contrôlé	13
5.2.3	Phase 2: Expérimentation du composant d'analyse statique	14
5.2.4	Phase 3: Expérimentation de la collecte de données en temps réel	14
5.2.5	Phase 4: Jeux de tests expérimentaux	15
5.2.6	Phase 5: Analyse critique et comparaison	15
5.2.7	Phase 6: Finalisation pour le rapport	15
6	Implémentation	16
6.1	Choix de Python	16
6.2	Structure du projet	16
6.3	Mesures de sécurité	17
6.3.1	Machine virtuelle dédiée	17
6.3.2	Réseau Docker isolé	17
6.3.3	Configuration du daemon Docker (daemon.json)	17
6.3.4	Limites de ressources au niveau du conteneur	18
6.4	Commandes CLI offertes	18
6.4.1	setup	18
6.4.2	analyze	18
6.4.3	scan-file	19
6.4.4	cleanup	19
6.5	Module dynamic.py	20
6.5.1	Chargement des règles (load_rules)	20
6.5.2	Détection des processus suspects (check_suspicious_processes)	20
6.5.3	Vérification du seuil CPU (check_cpu_limit)	20
6.5.4	Détection des ports suspects (check_suspicious_ports)	21
6.5.5	Calcul du CPU (formule Docker)	21
6.5.6	Collecte des connexions réseau	21

6.5.7	Collecte des logs	22
6.5.8	Flux d'exécution de <code>collect_dynamic_metrics</code>	22
6.6	Module <code>parser.py</code>	23
6.6.1	Détection du type de fichier (<code>detect_file_type</code>)	23
6.6.2	Extraction docker-compose (<code>extract_images_docker_compose</code>)	23
6.6.3	Extraction gitlab-ci (<code>extract_images_gitlab_ci</code>)	24
6.6.4	Point d'entrée (<code>extract_images</code>)	24
6.7	Module <code>static.py</code>	24
6.7.1	<code>collect_static_metrics</code>	24
6.7.2	<code>ioc_rules_reader</code>	25
6.7.3	<code>search_through_image_files</code>	25
6.8	Module <code>report.py</code> et <code>interface.py</code>	27
6.8.1	<code>report.py</code>	27
6.8.2	<code>interface.py</code>	27
6.9	Fichiers de test	28
6.9.1	fake-miner: Image malveillante simulée	28
6.9.2	docker-compose.yml de test	29
6.9.3	.gitlab-ci.yml de test	29
6.9.4	<code>run_tests.sh</code>	29
6.10	Intégration dans un pipeline CI/CD	34
6.10.1	Workflow typique	34
6.10.2	Prérequis pour l'intégration	34
6.10.3	Exemple d'intégration GitLab CI	34
6.10.4	Exemple d'intégration GitHub Actions	35
6.10.5	Avantages de l'approche par fichier yaml	36
6.10.6	Limites dans un contexte CI/CD	36
6.11	Résultats du projet académique	36
6.11.1	Résultats de l'analyse dynamique	36
6.11.2	Résultats de l'analyse statique	43
6.11.3	Réponse à la problématique	47
6.12	Difficultés et limites rencontrées	48
6.12.1	Pas de support GPU réel	48
6.12.2	Réseau isolé et tentatives bloquées	48
6.12.3	Durée d'observation fixe	48
6.12.4	Image de test personnalisée	48
6.12.5	Manque d'analyse en profondeur des fichiers	48
7	Conclusion	49
A	Contribution	52
A.1	Contributions des membres de l'équipe	52

1 Problématique à résoudre

1.1 Le crypto-jacking comme menace informatique moderne

La mine de crypto-monnaie, ou cryptomining en anglais, est la génération des pièces de crypto-monnaie et la validation des transactions faites dans un réseau blockchain pour mettre à jour le registre. Ceux qui font le minage de crypto-monnaie sont payés avec de la crypto-monnaie et comme ces monnaies digitales valent énormément d'argent, cette pratique est forcément très rentable [9].

Cependant, cette technologie a attiré l'attention d'acteurs malveillants. Les propriétaires de portefeuilles de crypto-monnaies restent anonymes et les transactions ne révèlent pas les identités des individus impliqués. Ainsi, cet anonymat permet aux cybercriminels de se faire de l'argent sans se faire prendre. C'est pour cette raison que les victimes du rançongiciel WannaCry devaient payer en bitcoin à un portefeuille digital [19]. De plus, comme cette technologie est très rentable, il est évident que cela attire les cybercriminels ayant un but de gain d'argent.

Ainsi, il n'est pas étonnant que des acteurs malfaisants cherchent à exploiter les machines de leurs victimes pour la génération de crypto-monnaies. Cela peut être de la forme de minage de crypto-monnaies sur la machine de leurs victimes, ou bien voler les pièces de crypto-monnaies déjà présentes. Cette pratique s'appelle le crypto-jacking. En cas légitime, une personne qui pratique le minage de crypto-monnaie mettrait en place son propre environnement avec du matériel acheté légalement, soit une salle de cartes graphiques avec un serveur de contrôle avec le système de refroidissement nécessaire entre autres.

1.2 Le cryptomining malveillant dans les environnements conteneurisés

Docker est la technologie de choix pour les développeurs. La conteneurisation permet à ces derniers de facilement déployer leurs applications dans un environnement qu'ils contrôlent et de les migrer avec aisance. De plus, les développeurs peuvent obtenir ou même publier leurs propres images de conteneur sur la plateforme Docker Hub, permettant ainsi la collaboration communautaire de tous les utilisateurs.

Cependant, les acteurs malfaisants ont aussi prêté attention à Docker Hub. Ces acteurs ont remarqué que la plateforme mettait beaucoup de confiance envers ses utilisateurs, laissant n'importe qui se créer un compte publier des images. De plus, Docker Hub ne vérifie pas individuellement la nature de l'image avant de la publier, ce qui explique la prolifération des images malveillantes. C'est ainsi qu'il y a eu des attaques de chaînes d'approvisionnement sous la forme de typosquatting, une pratique visant à ressembler le nom d'une image légitime pour tromper un utilisateur de télécharger une image malveillante.

En plus de cela, les images de conteneurs ont certainement des vulnérabilités logicielles dont des acteurs malveillants peuvent tirer avantage. Des étudiantes de l'université des sciences et technologies norvégiennes ont révélé en 2020 que la quantité de vulnérabilités introduites sur Docker Hub a augmenté avec les images de type certifiée ayant le plus de vulnérabilités [20]. Cette étude corrobore avec une autre étude indiquant que les images de base, c'est-à-dire des images seulement avec le système d'exploitation et le minimum de bibliothèques utilisées pour construire des images plus concrètes, était une source principale des vulnérabilités. Ces vulnérabilités persistent alors dans les images dérivées, ce qui augmente le nombre de conteneurs vulnérables à des acteurs malveillants [10].

1.3 Réalité du phénomène

Ce problème n'est pas un phénomène niche. En effet, l'organisation Sysdig a trouvé 1650 images de conteneurs ayant du malware sur la plate-forme Docker-Hub. Parmi les comportements malicieux, le crypto-jacking est le plus populaire, soit 608 images suivies par le type 281 ayant des secrets cachés [16] [1]. Le but est que des utilisateurs ou des compagnies téléchargent des images malicieuses, alors que ces derniers voulaient déployer facilement leurs applications. Ainsi, lorsque les utilisateurs déploient un conteneur, celui-ci se mettra à utiliser les ressources de la machine hôte pour miner ou voler de la crypto-monnaie.

Pour continuer, le site BleepingComputer a rapporté en 2021 que deux douzaines d’images malveillantes de crypto-jacking postées sur Docker Hub ont été téléchargées plus de 20 millions de fois[6]. Ces images malveillantes auraient pu avoir généré 200,000 USD de revenu pour leurs fournisseurs. Ainsi, cela est donc un moyen efficace de générer des profits sans investir dans leur propre environnement.

1.4 Conclusion

Les constats précédents mettent en évidence la réalité et la gravité du cryptomining malveillant au sein des images Docker provenant de registres publics. Malgré l’existence de solutions de sécurité largement adoptées dans les chaînes CI/CD, ces menaces continuent de se propager et d’échapper aux mécanismes de détection traditionnels.

Afin de mieux comprendre les limites actuelles de la protection des images conteneurisées, il est nécessaire d’analyser les travaux existants et les outils de référence utilisés pour la sécurisation des images Docker. Cette analyse a pour objectif de situer notre problématique par rapport aux solutions existantes, d’examiner les mesures de sécurité proposées, ainsi que leurs limites dans la détection des comportements de cryptomining malveillant, et de justifier les décisions techniques qui seront prises pour ce projet.

2 État de l’art

2.1 Problématique antérieure (et leur solution)

Voici un cas d’attaque correspondant à notre projet.

2.1.1 Cas de Uptycs:

Le groupe TeamTNT a déployé un conteneur à image malicieuse pour infecter des ordinateurs et les utiliser pour faire du cryptomining.

Méthode d’attaque

Plan de l’attaque:

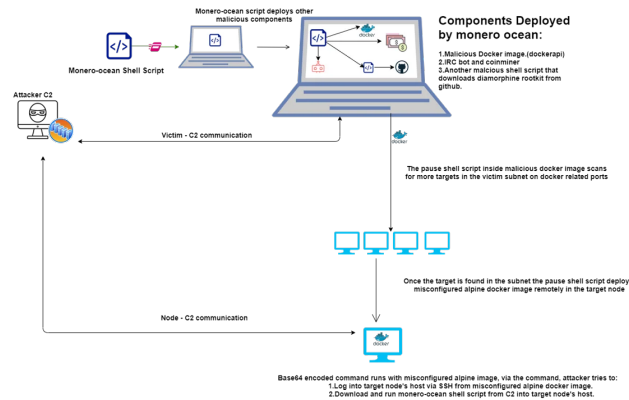


Figure 1: Plan d’attaque de uptyc

- En utilisant le script shell monero-ocean, TeamTNT/Hilde a déployé une nouvelle image Docker malveillante nommée Dockerapi, qui était hébergée sur le site Docker Hub.
- En utilisant Docker, l’image malveillante a été exécutée avec le drapeau de privilège et a été montée avec l’hôte de la victime et la configuration réseau de l’hôte de la victime.

- L'image Docker malveillante contenait un script shell intégré nommé «pause»
- Le script shell « pause » à l'intérieur de l'image Docker malveillante contenait des commandes pour installer masscaner et l'outil zgrab.
- Après avoir mis en place les outils de scan, les fonctions du script « pause » commencent à scanner rigoureusement le sous-réseau de la victime sur les ports liés à Docker pour trouver d'autres machines virtuelles cibles (nœuds). Un nœud fait partie d'un swarm Docker. Un swarm Docker est un groupe de machines physiques ou virtuelles (nœuds) opérant en cluster.
- Une fois que le nœud cible est trouvé à la suite du scan des ports liés à Docker dans le sous-réseau de la victime, le script shell pause exécute à distance l'image Docker alpine mal configurée (depuis la machine de la victime) sur le nœud cible, en passant une commande en base64 en ligne de commande.

Détection

Aucune méthode de détection n'est mentionnée dans cet article. Bien que les conseils offerts par le site soient utilisés pour vendre leurs produits, on peut sécuriser les conteneurs en installant des outils CI/CD pour scanner des images avant qu'ils ne soient déployés dans la production. On peut aussi surveiller et analyser en lien avec la CIS référence pour détecter des problèmes de configuration.

2.2 Analyse des outils existants

En résumé, les conteneurs Docker sont alors des cibles d'attaques par des acteurs malveillants. Que ce soit par le cryptojacking, l'attaque de chaîne d'approvisionnement ou le piratage direct, les conteneurs sont à risque pour les organisations. Cela a alors motivé la création d'outils d'analyse des images de conteneurs pour déterminer le risque posant aux organisations. Il en existe plusieurs, mais nous allons nous concentrer sur deux en particulier: Trivy de l'organisation Aqua et Docker Scout étant l'outil d'analyse intégré sur Docker Desktop.

2.2.1 Aqua Trivy

Trivy est un outil d'analyse d'images de conteneurs développé par l'organisation Aqua. Cet outil offre de multiples moyens d'installation, dont notamment dans un conteneur Docker dédié que nous avons pris pour le tester [18].

Cet outil fonctionne en analysant les fichiers dans l'image. D'abord, elle charge la base de données des vulnérabilités (CVE, CWE), puis identifie les bibliothèques, les dépendances et les binaires des logiciels installés ainsi que leurs versions. Ensuite, il consulte la base de données pour déterminer si les logiciels installés ont des vulnérabilités trouvées. L'outil collecte et analyse les informations et présente à l'utilisateur toutes les vulnérabilités avec leur sévérité, la version du logiciel présent et s'il y a un patch pour le résoudre. Les utilisateurs pourront alors prendre des décisions sur comment gérer ces vulnérabilités, ou même s'ils veulent encore utiliser l'image [18].

L'outil peut aussi détecter des erreurs de configuration dans l'image du conteneur, plus précisément dans des scripts IaC, et dans les métadonnées. Les secrets exposés comme des mots de passe, des clés d'API, des jetons, des variables d'environnement ou autres données sensibles, ainsi que les licences seront détectés si l'option est activée [18].

Trivy peut consulter le Software Bill of Materials (SBOM), un inventaire généré par le fournisseur de l'image contenant les noms et versions de toutes les bibliothèques, les dépendances et les composantes. Avec cela, il peut mener son analyse des vulnérabilités [18].

2.2.2 Docker Scout

Docker Scout est similaire à Trivy, sauf que ce dernier se contente d'analyser les métadonnées et le Software Bill of Materials (SBOM). Docker Scout vérifie si chacune d'entre elles a des CVE ou des CWE associés

publiés avec le score CVSS et EPSS. De plus, cet outil est intégré dans l'interface d'utilisateur de Docker Desktop, demandant aucune autre installation pour l'utilisateur [7].

2.2.3 Atouts de ces outils

Ces outils sont relativement simples à utiliser et produisent des rapports assez détaillés pour les utilisateurs. Les utilisateurs démarrent Docker Scout en sélectionnant l'image à analyser et en appuyant sur le bouton. Ils obtiennent alors la liste de vulnérabilités associées à chaque dépendance avec la criticité en plus d'un code de couleur indiquant celle-ci. Aqua Trivy requiert certes une installation à part entière, mais les utilisateurs peuvent simplement télécharger l'image de conteneur dédié à l'outil, démarrer un conteneur et exécuter l'analyse des images.

Un autre atout est dans leur intégration dans le cycle de développement. En effet, les développeurs peuvent exécuter Aqua Trivy sur les images avant le déploiement des conteneurs. Ainsi, cela peut réduire le vecteur d'attaque lors de la phase de développement et de déploiement

2.2.4 Limites de ces outils

La limite principale de Docker Scout est sur la génération de l'attestation du SBOM dépend du fournisseur. En effet, la responsabilité de sa génération et de son intégrité est au fournisseur, et donc peut être soit falsifiée, soit désuète ou même soit manquer des dépendances lors de sa génération. De plus, elle peut être modifiée en cours de chemin et donc requiert une signature pour la valider.

La limite s'appliquant à ces outils d'analyse d'image est leur portée. En effet, le but de Trivy, Docker Scout, Clair, et les autres est de déterminer si les dépendances, le système d'exploitation ou les bibliothèques présentes ont des vulnérabilités qui ont été trouvées. Or, cela n'aide pas à découvrir:

- Des vulnérabilités Zero-day peuvent être présentes sans que le fournisseur de l'image ou les utilisateurs le sachent.
- Des virus ou du contenu malveillant compromis dans l'image du conteneur.
- Du malware comme du rançongiciel ou du crypto-jacking.

Ainsi, il y a alors un vide à combler pour détecter des images ayant du contenu malveillant caché. Ces outils, n'étant pas fait pour cette tâche, ne peuvent pas contrecarrer les techniques de crypto-jacking et la menace continue de s'amplifier. Les meilleurs moyens disponibles seraient préventifs, soit de vérifier que le fournisseur de l'image est légitime, soit d'utiliser des signatures des images pour confirmer sa légitimité ou lancer des conteneurs dans des environnements isolés.

2.3 Indicateurs utiles

Afin d'implémenter notre projet, il est important de connaître les indicateurs de cryptomining que ce soit sur l'application ou l'ordinateur en lui-même. Cela permettra de pouvoir développer un outil d'analyse dynamique et si donner le temps, une d'analyse statique.

Analyse statique (image Docker):

- Présence de binaires connus:
 - XMRig (51% en 2021 selon L'ENISA) [8] [3] [21]
 - MinerD [3] [4]
- Dockerfile suspect:
 - curl, sh
 - wget

- Permissions excessives
- Scripts shell
- URLs de pools de minage de cryptomonnaies

Analyse dynamique:

- Ouverture de terminal à l'exécution
- Utilisation CPU/GPU anormale [3]
- Processus cachés
- Redémarrage automatique
- Connexions réseau persistantes [4]
- Environnement sécurisé et isolé pour rouler ces environnements

Le problème majeur de l'analyse dynamique pour notre projet est le manque de référence permettant de guider nos choix pour faire une analyse pendant l'exécution d'une image de docker. Nous chercherons alors des indicateurs potentiels venant de la source de revenu des crypto jackings, les machines pour miner de la cryptomonnaie.

La première sera la surveillance du réseau pour détecter les connexions et voir si elles accèdent à des sites de cryptomining connues ou des serveurs inconnus. Il faudra aussi regarder dans le cas de manque de connexion à l'internet car les systèmes de crypto mining nécessitent une connexion constante pour marcher ce qui peut révéler un malware si l'ordinateur tente des requêtes internet sans être sollicité.

3 Retour sur la Problématique

Les images Docker issues de registres publics peuvent intégrer des comportements malveillants, notamment du cryptomining clandestin, qui ne sont pas détectés efficacement par les outils de sécurité actuels reposant principalement sur l'analyse statique et des bases de vulnérabilités connues.

Dans quelle mesure une approche combinant l'analyse statique et l'analyse dynamique du comportement à l'exécution permet-elle d'identifier de manière plus fiable des images Docker impliquées dans des activités de cryptomining malveillant, tout en limitant les faux positifs, par rapport aux solutions existantes?

Ensuite, les opérations de crypto mining nécessitent des appareils puissants pour le fonctionnement pour les calculs ce qui peut démontrer la présence d'un malware si l'utilisation du GPU/CPU grimpe sans l'utilisation d'une application consommant autant.

Une analyse des fichiers et programmes serait utile pour faire la détection de programmes cachés dans l'appareil.

Une ouverture soudaine du terminal au téléchargement est incriminant de la présence d'un malware au même point qu'une demande de redémarrage de l'ordinateur. Les deux peuvent apporter des modifications sur l'appareil au dépend de l'utilisateur.

En conclusion, l'analyse dynamique va reposer sur regarder le réseau et le comportement de l'appareil ainsi que le code des liens pour regarder. De par la dangerosité de l'utilisation de tel outil, un environnement virtuel est nécessaire pour accomplir des tests sans danger pour l'appareil de test.

4 Justification du projet et délimitation de son périmètre

La problématique de la sécurité des images Docker s’inscrit dans un contexte plus large de détection de logiciels malveillants. En effet, une image Docker peut intégrer diverses formes de comportements malveillants, tels que des mécanismes d’exfiltration de données, des portes dérobées, des charges de rançongiciel ou encore des activités frauduleuses dissimulées.

Toutefois, le projet présenté ici ne vise pas à proposer une solution générale capable de détecter l’ensemble des malwares susceptibles d’être intégrés dans une image Docker. Un tel objectif dépasserait largement le cadre expérimental d’un projet académique et nécessiterait des ressources techniques et d’infrastructures considérables. Nous avons donc fait le choix d’une délimitation volontaire du périmètre de projet.

Notre travail se concentre spécifiquement sur une sous-catégorie de comportements malveillants: le cryptomining exécuté au sein de conteneurs Docker. Ce choix est motivé par plusieurs facteurs. Premièrement, le cryptomining présente des caractéristiques comportementales observables et mesurables, telles que des utilisations soutenue et continue des ressources CPU ou GPU, des connexions réseau persistantes vers des pools de minage, ou encore l’exécution de processus spécifiques. Ces éléments constituent des indicateurs de compromission (IoC) relativement bien documentés par la communauté en sécurité logicielle.

Deuxièmement, les outils et binaires associés au cryptomining, tels que XMRig ou MinerD, sont largement connus et fréquemment identifiés dans les environnements conteneurisés compromis. Cette documentation existante permet de définir des critères d’analyse précis, reproductibles, et exploitables dans un cadre expérimental.

Ce positionnement se distingue des outils d’analyse d’images Docker existants, qui se concentrent principalement sur la détection de vulnérabilités logicielles, d’erreurs de configuration ou de secrets mal protégés. Ces outils s’appuient souvent sur un SBOM afin d’identifier les composants logiciels présents et signaler les versions vulnérables, dans le but d’améliorer la gestion des mises à jour et de réduire la surface d’attaque. Or, dans notre cas, ces approches ne répondent pas directement à la problématique étudiée, à savoir la détection d’images malveillantes intégrant des mineurs de crypto-monnaies tout en se présentant comme des images légitimes, souvent largement diffusées et utilisées.

Ainsi, nous proposons le développement d’un prototype orienté vers la détection de bibliothèques, de binaires et de comportements associés au cryptomining dissimulé dans des images Docker apparemment légitimes. Ce prototype repose sur deux axes principaux: d’une part, une analyse heuristique du comportement du conteneur, notamment sur l’utilisation anormale des ressources de la machine hôte (CPU, GPU et mémoire), et d’autre part, l’exécution des images dans un environnement contrôlé, permettant une observation sécurisée sans impact sur le système hôte.

En restreignant volontairement le périmètre du projet, cette approche permet de mener une analyse plus approfondie et de bien évaluer la capacité de détection proposée. Ce travail vise à contribuer à la détection du cryptomining obfusqué dans des environnements conteneurisés et à limiter l’exploitation abusive des ressources de l’hôte par des acteurs malveillants.

5 Définition de l’implémentation

5.1 Squelette de l’architecture de l’outil

Cette section présente le squelette de l’architecture de l’outil proposé. Elle décrit les composants principaux du système et la manière dont ils interagissent afin de permettre l’analyse et la détection de comportements de cryptomining dans des images Docker.

L’objectif de l’outil est de permettre à un utilisateur ou une organisation de fournir une image Docker à

l'outil d'analyse. L'outil va effectuer une analyse de cette image et selon le résultat de cette analyse va classer l'image comme étant légitime, suspecte ou potentiellement malveillante.

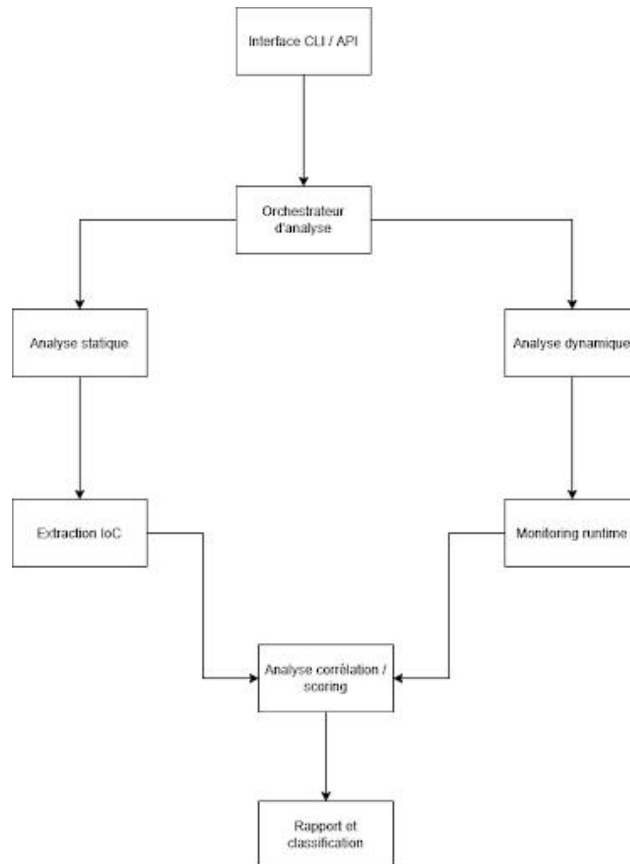


Figure 2: Squelette de l'architecture de l'outil

Interface CLI / API Point d'entrée de l'outil. Doit :

- Charger l'image Docker
- Déclencher l'analyse
- Afficher les résultats

Orchestrateur d'analyse Coordonne les différentes phases :

- Analyse statique
- Analyse dynamique
 - Lancement de l'environnement isolé
- Accumulation des résultats

Analyse statique

- Inspection des couches de l'image
- Lecture du Dockerfile
- Génération d'un SBOM
- Extraction des binaires
- Analyse des permissions

- Recherche de chaînes suspectes (pools, wallets)

Extraction des IoC Détecte :

- Présence de XMRig / MinerD
- URLs de pools de minage
- Commandes suspectes
- Scripts shell persistants
- Autres indicateurs

Analyse dynamique Exécution du conteneur :

- Dans un environnement contrôlé
- Avec limites imposées sur le réseau
- Monitoring actif

Monitoring runtime Mesure :

- Utilisation CPU
- Processus actifs
- Connexions réseau
- Mécanismes de persistance
- Tentatives de redémarrage
- Autres comportements

Analyse de corrélation / scoring

- Corrélation entre analyse statique et dynamique
- Utilisation d'un score pondéré
- Détermination des seuils de classification

Rapport et classification

- Score final
- IoC détectés
- Logs d'exécution
- Classification
- Export dans un format structuré (JSON, CSV, etc.)

5.1.1 Principes généraux

L'architecture proposée s'inspire des modèles classiques d'analyse de malwares décrits par le NIST [15] [14] et des approches comportementales présentées dans la littérature académique [5] [2].

L'objectif est de concevoir un outil expérimental, permettant:

- la collecte d'indicateurs mesurables,
- l'analyse comportementale ciblée sur le cryptomining,
- la corrélation d'IoC (Indicators of Compromise),
- la génération d'un verdict explicable.

Le modèle ATT&CK de MITRE [13] sert de référence pour structurer les comportements observables et positionner les techniques analysées.

5.1.2 Architecture globale proposée

L'outil s'organise autour de cinq modules principaux:

Acquisition des données

Ce premier module est dédié à la collecte des données pertinentes sur le comportement du conteneur lorsqu'il est exécuté dans un environnement isolé. Ce module mesure notamment l'utilisation des ressources CPU et mémoire, les tentatives de connexions réseau, les processus actifs et autres. L'importance de ce type de collecte est documentée par des travaux montrant que les campagnes de cryptomining présentent des signatures comportementales répétitives telles qu'une forte utilisation des ressources ou des connexions persistantes vers des pools de minage [11] [8] [12] [17].

Analyse statique

L'analyse statique consiste à inspecter le contenu de l'image sans l'exécuter. Elle inclut l'extraction des métadonnées, des couches Docker et l'inspection des fichiers pour identifier des chaînes suspectes (telles que des adresses de pools de minage ou des binaires associées à des mineurs connus). Puisque ce type d'outils existe déjà sur le marché via des solutions gratuites et propriétaires, cet aspect de l'outil sera mis de côté pour notre implémentation expérimentale. Pour une question de temps et de ressources, le module sur l'acquisition des données et celui de l'analyse comportementale seront proposés lors du développement.

Analyse comportementale

Le coeur de l'outil repose sur l'analyse comportementale dynamique, réalisée en exécutant l'image dans un environnement isolé. L'outil observe, en temps réel, les paramètres d'exécution (utilisation CPU/GPU, connexions réseau, processus en cours) afin de détecter des motifs associés au cryptomining. Cette approche est soutenue par des travaux récents qui utilisent des techniques eBPF et l'apprentissage supervisé pour surveiller et classer des comportements de conteneurs en fonction de leurs activités à l'exécution [12].

Corrélation d'indicateurs et scoring

Les résultats des modules d'analyse statique et dynamique sont ensuite envoyés dans un moteur de corrélation qui applique un système de scoring. Inspiré des approches de détection comportementale, ce modèle de scoring pondère les indicateurs observés afin de réduire les faux positifs, un enjeu reconnu dans la détection d'intrusions et comportements malveillants [2]. Par exemple, une forte utilisation CPU jumelée à des connexions réseau vers des pools listés comme suspects peut augmenter le score d'une possible image compromise.

Rapport

Enfin, l'outil génère un rapport structuré détaillant les indicateurs observés, le score final de classification et les éléments justificatifs associés. Cette traçabilité correspond aux recommandations de bonnes pratiques en matière de gestion des incidents de sécurité: fournir un verdict explicable et une justification des décisions prises par l'outil.

5.2 Plan de projet

Objectif de l'implémentation:

Mettre en œuvre un prototype fonctionnel capable d'exécuter une image Docker dans un environnement contrôlé et d'observer automatiquement des comportements caractéristiques du cryptomining, afin de :

- valider les hypothèses issues de l'analyse,
- démontrer la faisabilité de l'approche comportementale,
- comparer qualitativement les résultats avec des outils existants.

5.2.1 Phase 0: Cadrage expérimentale

Objectifs:

- Délimiter précisément ce qui sera implémenté et ce qui ne le sera pas.
- Transformer les IoC théoriques en données mesurables.

Tâches:

- Sélection des comportements observables
- Définition des métriques
- Choix des outils de monitoring
- Définition du format de sortie

Livrables:

- Liste des IoC exploitables
- Listes des techniques de monitoring

5.2.2 Phase 1: Mise en place de l'environnement d'exécution contrôlé

Objectifs:

- Garantir un environnement isolé, reproductible et sécurisé.
- Pouvoir exécuter des images potentiellement malveillantes sans risque

Tâches:

- Création d'un hôte d'analyse dédié
 - VM
 - Docker installé et configuré
- Configuration de l'exécution sécurisée
- Script d'orchestration

- Lancer le conteneur
- Démarrer la capture des métriques
- Arrêter proprement après un temps donné

Livrables:

- Scripts d'exécution
- README d'installation
- Configurations de sécurité de l'environnement isolé

5.2.3 Phase 2: Expérimentation du composant d'analyse statique

L'analyse statique est déjà un outil en place accessible à tous. Le but ici est de déterminer si, selon le temps fourni, l'implémentation de ce composant dédié à l'analyse statique de cryptojacking est pertinente.

Objectifs:

- Déterminer la complexité et le besoin d'implémenter le composant d'analyse statique
- Mesurer la valeur ajoutée par rapport aux outils existants

Tâches:

- Identifier les techniques actuelles d'analyse statique pour la détection de:
 - code malveillant
 - scripts de minage
- Identifier les indicateurs statiques détectables
- Réaliser un prototype minimal de détection
- Analyser la pertinence et les limites

Livrables:

- Prototype expérimental minimal

5.2.4 Phase 3: Expérimentation de la collecte de données en temps réel

Objectifs:

- Observer le comportement réel du conteneur pendant son exécution.
- Collecter des données exploitables pour l'analyse.

Tâches:

- Monitoring des ressources
- Observation des processus
- Analyse réseau
- Journalisation

Livrables:

- Code
- Tests simples
- Exemples de journalisation

5.2.5 Phase 4: Jeux de tests expérimentaux

Objectifs:

- Évaluer l'outil sur des cas réels
- Produire des résultats analysables dans le rapport

Tâches:

- Sélection d'images légitimes
- Sélection d'images malveillantes (ou simulées)
- Exécution des campagnes de test
- Collecte et comparaison des résultats

Livrables:

- Scripts de test

5.2.6 Phase 5: Analyse critique et comparaison

Objectifs:

- Mettre en perspective les résultats
- Relier l'implémentation et la problématique

Tâches:

- Analyse de faux positifs / faux négatifs
- Comparaison qualitative avec outils existants
- Discussion des limites

5.2.7 Phase 6: Finalisation pour le rapport

Objectifs:

- Transformer le travail technique en travail de recherche

Tâches:

- Description claire de l'architecture implémentée
- Justification des choix techniques
- Discussion critique
- Perspectives d'amélioration

6 Implémentation

6.1 Choix de Python

L'outil s'organise autour de cinq modules principaux:

SDK Docker officiel: Le projet 'docker' (pip) offre une interface Python complète et bien documentée pour interagir avec le daemon Docker : lancer des conteneurs, lire les stats en temps réel, récupérer les logs, inspecter les processus. C'est l'outil le mieux adapté à notre besoin sans passer par des appels shell manuels.

Rapidité de prototypage: Python permet d'itérer rapidement sur les règles de détection et la logique de scoring sans overhead de compilation. Pour un projet de recherche académique avec des contraintes de temps, c'est un avantage décisif.

Écosystème riche: Les bibliothèques 'pyyaml' (lecture des règles IoC), 'typer' (CLI), et 'concurrent.futures' (analyse parallèle) sont disponibles et à jour.

Lisibilité: Le code Python est accessible à tous les membres de l'équipe peu importe leur background, facilitant la collaboration.

6.2 Structure du projet

Listing 1: Structure du projet

```
1  cryptojack-detector/
2  |-- main.py                # Point d'entree CLI
3  |-- analyzer/
4  |   |-- __init__.py
5  |   |-- dynamic.py        # Collecte des metriques runtime
6  |   |-- parser.py         # Collecte les images dans les fichiers
7  |                           .gitlab-ci.yml et docker-compose.yml
8  |   |-- static.py         # Analyse statique
9  |   |-- scoring.py        # Correlation et scoring
10 |   |-- interface.py       # Dataclass pour la generation des rapports
11 |   |-- report.py          # Generation de rapports
12 |-- config/
13 |   |-- ioc_rules.yaml    # Regles IoC configurables
14 |-- output/
15 |   |-- test_report.json  # Exemple rapport json
16 |-- tests/
17 |   |-- run_tests.sh      # Script pour lancer les tests
18 |   |-- test_report.py    # Exemple generation rapport json
19 |   |-- test_images/
20 |       |-- fake_miner/
21 |           |-- Dockerfile # Image malveillante simulée
22 |           |-- docker-compose.yml # Fichier de test docker-compose
23 |           |-- .gitlab-ci.yml # Fichier de test gitlab-ci
24 |-- cryptojack-detector.sh # Script de configuration réseau
25 |                           isolé et du Docker daemon
26 |-- .gitignore
27 |-- daemon.json          # Configuration du daemon de Docker
28 |-- requirementstxt      # Fichier pour les packages de l'env python
29 |-- README.md
```

Séparation des responsabilités: Chaque module a une responsabilité unique et claire. 'dynamic.py' et 'static.py' collecte, 'scoring.py' évalue, 'report.py' présente, 'parser.py' extrait. Cette séparation facilite le

travail en parallèle entre les membres de l'équipe et rend le code testable indépendamment.

Configuration externe: Les règles IoC sont dans 'ioc.rules.yaml' plutôt que dans le code. Cela permet de mettre à jour les règles (ajouter un nouveau binaire suspect, changer un seuil CPU) sans toucher au code Python.

CLI avec Typer: L'interface en ligne de commande expose trois commandes : 'setup' pour initialiser l'environnement, 'analyze' pour analyser une image spécifique, 'scan-file' pour analyser toutes les images d'un fichier docker-compose ou gitlab-ci, et 'cleanup' pour détruire l'image si celle-ci se trouve être problématique. Cette structure est pensée pour l'intégration dans un pipeline CI/CD.

6.3 Mesures de sécurité

L'exécution d'images potentiellement malveillantes requiert un environnement rigoureusement sécurisé. Plusieurs couches de protection ont été mises en place.

6.3.1 Machine virtuelle dédiée

L'outil tourne sur une VM VirtualBox dédiée exclusivement à l'analyse, séparée physiquement de la machine de développement. Si un conteneur réussit à s'échapper, il n'atteint que la VM, pas la machine hôte réelle.

6.3.2 Réseau Docker isolé

Un réseau Docker nommé 'cryptojack-timeout' est créé avec les options suivantes :

Listing 2: création du réseau isolé

```
1 docker network create \  
2   --driver bridge \  
3   --internal \  
4   --subnet 192.168.100.0/24 \  
5   cryptojack-timeout
```

L'option '-internal' est critique : elle supprime la route par défaut vers internet depuis les conteneurs. Un malware qui tente de contacter un pool de minage ne peut pas établir la connexion. Le trafic réseau reste cependant observable (tentatives de connexion) via les stats Docker.

6.3.3 Configuration du daemon Docker (daemon.json)

Listing 3: fichier daemon.json

```
1 {  
2   "no-new-privileges": true,  
3   "usersns-remap": "default",  
4   "log-driver": "json-file",  
5   "log-opts": {  
6     "max-size": "10m",  
7     "max-file": "3"  
8   }  
9 }
```

'no-new-privileges' empêche tout processus dans le conteneur d'obtenir des privilèges supplémentaires via sudo ou setuid. 'usersns-remap' remappe les UIDs du conteneur vers des UIDs non-privilégiés sur l'hôte — un processus root dans le conteneur n'est pas root sur la VM.

6.3.4 Limites de ressources au niveau du conteneur

Chaque conteneur analysé est lancé avec des limites strictes :

Listing 4: programme de lancement du conteneur

```
1 container = client.containers.run(  
2     image_name,  
3     detach=True,  
4     network=network,  
5     mem_limit="512m", # Maximum de 512Mb de memoire  
6     nano_cpus=1_000_000_000, # Maximum 1 CPU  
7     remove=False,  
8 )
```

Ces limites préviennent qu'un malware monopolise toutes les ressources de la VM pendant l'analyse.

6.4 Commandes CLI offertes

L'outil expose quatre commandes via 'main.py'. Toutes les commandes s'exécutent depuis la racine du projet avec le venv activé.

6.4.1 setup

Configure l'environnement isolé nécessaire à l'analyse.

Listing 5: commande setup

```
1 python main.py setup
```

Lance le script 'cryptojack-detector.sh' qui crée le réseau Docker isolé 'cryptojack-timeout' si celui-ci n'existe pas déjà. Cette commande doit être exécutée une seule fois avant la première analyse, ou après une réinstallation de Docker.

6.4.2 analyze

Analyse une image Docker unique pour détecter des comportements de cryptomining.

Listing 6: commande analyze

```
1 # Minimal  
2 python main.py analyze --image nginx:latest  
3 # Complet  
4 python main.py analyze --image nginx:latest  
5                         --duration 60 --interval 5  
6                         --output ./rapports  
7 # Avec raccourcis  
8 python main.py analyze -i nginx:latest -d 60
```

Option	Raccourci	Défaut	Description
- -image	-i	Obligatoire	Nom de l'image Docker à analyser
- -duration	-d	30	Durée de l'observation en secondes
- -interval		2	Intervalle entre chaque mesure en secondes
- -output	-o	./output	Dossier de sortie du rapport JSON

Table 1: Choix d'options pour l'analyse

L'image est d'abord recherchée localement. Si elle est absente, elle est téléchargée depuis Docker Hub. Le conteneur est lancé dans le réseau isolé avec des limites de ressources (512 MB RAM, 1 CPU). Les métriques sont collectées en temps réel pour l'analyse dynamique puis exportées dans un fichier JSON nommé '{image}_{tag}_dynamic.json'. Pour l'analyse statique, les données sont collectées dans les couches de l'image de conteneur et du système de fichiers d'un conteneur créé à partir de l'image. Ces données sont traitées puis exportées dans un fichier JSON nommé '{image}_{tag}_static.json'.

6.4.3 scan-file

Extrait toutes les images d'un fichier 'docker-compose.yml' ou '.gitlab-ci.yml' et les analyse en parallèle.

Listing 7: commande extraction images

```

1 # docker-compose
2 python main.py scan-file --file docker-compose.yml
3
4 # gitlab-ci
5 python main.py scan-file --file .gitlab-ci.yml --duration 60
6
7 # Avec raccourcis
8 python main.py scan-file -f docker-compose.yml -d 30

```

Option	Raccourci	Défaut	Description
- -file	-f	Obligatoire	Chemin vers le fichier yaml à analyser
- -duration	-d	30	Durée d'observation par image en secondes
- -interval		2	Intervalle entre chaque mesure en secondes
- -output	-o	./output	Dossier de sortie du rapport JSON

Table 2: Choix d'options pour l'extraction

Le type de fichier est détecté automatiquement. Pour un 'docker-compose.yml', les images sont extraites de chaque service. Pour un '.gitlab-ci.yml', l'image globale et les images par job sont extraites. Un rapport JSON séparé est généré pour chaque image analysée.

6.4.4 cleanup

Supprime une image Docker de la machine après analyse.

Listing 8: commande nettoyage

```

1 python main.py cleanup --image fake-miner:latest
2
3 # Avec raccourci
4 python main.py cleanup -i fake-miner:latest

```

Option	Raccourci	Défaut	Description
- -image	-i	Obligatoire	Nom de l'image Docker à supprimer

Table 3: Choix d'options pour le nettoyage

Utile après l'analyse d'une image identifiée comme malveillante: permet de s'assurer qu'elle ne reste pas sur la machine. Utilise 'force=True' pour supprimer même si des conteneurs arrêtés référencent encore l'image.

6.5 Module dynamic.py

dynamic.py est une des parties les plus importantes de l'outil. Il contient toute la logique de collecte des métriques runtime et de détection des IoC dynamiques. La fonction principale `collect_dynamic_metrics` orchestre l'ensemble du cycle : lancement du conteneur, collecte des métriques, détection des IoC, arrêt propre et retour d'un objet `IOC_Dynamic`.

6.5.1 Chargement des règles (load_rules)

Listing 9: programme de chargement des règles

```
1 def load_rules(rules_path: str = "config/ioc_rules.yaml") -> dict:
```

Charge le fichier `ioc_rules.yaml` au démarrage de chaque analyse. Si le fichier est introuvable, retourne un dictionnaire vide et affiche un avertissement. L'analyse continue mais aucun IoC ne sera détecté. Le fichier est encodé en latin-1 pour éviter les erreurs de décodage sur les caractères accentués présents dans les commentaires yaml.

La structure du fichier yaml attendue est :

Listing 10: Structure fichier yaml

```
1 rules:
2   ioc-binaire:
3     indicateur: [xmrig, minerd, kworker]
4   ioc-dynamic-core-limit:
5     indicateur:
6       - cpu: 70
7   ioc-persistent-connection:
8     indicateur:
9       port-suspicieux: [3333, 4444, 5555]
```

Toutes les fonctions de détection accèdent aux règles via `'rules.get("rules", )'`: la clé racine `rules` est obligatoire dans le yaml.

6.5.2 Détection des processus suspects (check_suspicious_processes)

Listing 11: Programme de détection des processus suspects

```
1 def check_suspicious_processes(processes: list, rules: dict) -> list:
```

Compare les noms des processus actifs dans le conteneur avec la liste des binaires suspects définie dans `ioc-binaire`. Utilise un set pour dédupliquer les résultats: si `kworker` tourne en deux instances simultanées, il n'est rapporté qu'une fois dans les IoC hits.

La comparaison est insensible à la casse (`p.lower()`) et cherche le nom du binaire n'importe où dans la ligne de processus. Ainsi, `/bin/sh /usr/bin/kworker` sera bien détecté même si la règle contient seulement `kworker`. La fonction retourne les noms des binaires suspects trouvés plutôt que les lignes de processus complètes, ce qui produit un résultat plus lisible dans le rapport JSON.

Cette vérification est effectuée à chaque itération de la boucle de collecte. Les résultats sont dédupliqués globalement à la fin via `list(set(ioc_hits))`.

6.5.3 Vérification du seuil CPU (check_cpu_limit)

Listing 12: Programme de vérification du seuil CPU

```
1 def check_cpu_limit(cpu_percent: float, num_cpus: int, rules: dict) -> bool:
```

6.5.4 Détection des ports suspects (check_suspicious_ports)

Listing 13: Programme de détection des ports suspects

```
1 def check_suspicious_ports(connections: list, rules: dict) -> list:
```

Cherche les ports associés au cryptomining dans les données réseau collectées. Les ports 3333, 4444 et 5555 sont les ports standards du protocole Stratum utilisé par les pools de minage de cryptomonnaie. Ces ports sont entièrement configurables dans `ioc_rules.yaml`: il suffit d'en ajouter ou d'en retirer sans modifier le code.

La détection cherche la chaîne `{port}` dans les données de connexion. Cette vérification est effectuée à chaque itération de la boucle.

6.5.5 Calcul du CPU (formule Docker)

Listing 14: Calcul du cpu

```
1 cpu_delta = (  
2     stats["cpu_stats"]["cpu_usage"]["total_usage"] -  
3     stats["precpu_stats"]["cpu_usage"]["total_usage"]  
4 )  
5 system_delta = (  
6     stats["cpu_stats"]["system_cpu_usage"] -  
7     stats["precpu_stats"]["system_cpu_usage"]  
8 )  
9 num_cpus = stats["cpu_stats"].get("online_cpus", 1)  
10  
11 if system_delta > 0:  
12     cpu_percent = min((cpu_delta / system_delta)  
13                       * num_cpus * 100.0, 100.0)  
14 else:  
15     cpu_percent = 0.0
```

C'est la formule officielle Docker pour calculer le pourcentage CPU. Elle compare la consommation CPU du conteneur entre deux snapshots successifs (`cpu_delta`) avec la consommation totale du système sur la même période (`system_delta`). Le ratio est multiplié par `num_cpus` pour obtenir un pourcentage sur l'ensemble des coeurs disponibles.

Le `min(..., 100.0)` plafonne la valeur à 100% pour éviter des pourcentages aberrants causés par des variations de timing entre les deux snapshots: ce problème a été observé en environnement VM où le `system_delta` peut être anormalement petit, produisant des ratios supérieurs à 100%.

Le guard `if system_delta > 0` évite une division par zéro si les deux snapshots sont identiques (peut arriver au premier appel).

6.5.6 Collecte des connexions réseau

Les connexions réseau sont collectées via les stats Docker plutôt que via `ss` ou `netstat` dans le conteneur. Cette approche est plus robuste car elle fonctionne même sur des images minimalistes qui n'ont pas ces outils installés (comme `nginx:alpine`): la tentative d'utiliser `ss` sur une image `nginx Alpine` a produit des erreurs `executable file not found in $PATH`.

Listing 15: Calcul du cpu

```

1 net_stats = stats.get("networks", {})
2 connections = [
3     f"{iface}: rx={data['rx_bytes']}B tx={data['tx_bytes']}B"
4     for iface, data in net_stats.items()
5 ]

```

Les données retournées par Docker Stats indiquent les bytes reçus (rx_bytes) et envoyés (tx_bytes) par interface réseau depuis le démarrage du conteneur. Ce sont des valeurs cumulatives: leur évolution entre les itérations permet d'observer si le conteneur tente de communiquer avec l'extérieur.

6.5.7 Collecte des logs

Listing 16: Collecte des logs

```

1 finally:
2     container.stop()
3     all_logs = collect_logs(container) # Avant remove()
4     container.remove()

```

Les logs sont collectés après l'arrêt du conteneur, juste avant sa suppression. Cette séquence est critique: Docker supprime les logs avec le conteneur si `remove=True` est passé au lancement, d'où l'utilisation de `remove=False` au démarrage et d'un `container.remove()` manuel après la collecte.

La collecte dans le bloc `finally` garantit que les logs sont toujours récupérés même si une exception survient pendant la collecte des métriques.

Les logs capturent l'ensemble des sorties `stdout/stderr` pendant toute la durée d'exécution. Ils sont ensuite analysés pour détecter des tentatives d'utilisation d'outils de téléchargement (`curl`, `wget`) ou d'exécution de scripts shell suspects. Un filtre évite les faux positifs sur les extensions `.sh`: les logs nginx contenant des références à des scripts d'initialisation comme `/docker-entrypoint.sh` déclenchaient incorrectement la règle `sh`.

6.5.8 Flux d'exécution de `collect_dynamic_metrics`

Listing 17: Collecte des metriques

```

1 1. Chargement des règles IoC depuis ioc_rules.yaml
2 2. Vérification de l'image (locale ou pull Docker Hub)
3 3. Lancement du conteneur dans le réseau isolé avec limites
4     de ressources
5 4. Boucle de collecte (répétée toutes les 'interval' secondes
6     pendant 'duration' secondes) :
7     a. Récupération des stats Docker (CPU, mémoire, réseau)
8     b. Calcul CPU normalisé et plafonné à 100%
9     c. Collecte des processus actifs via docker top
10    d. Vérification des processus suspects (ioc-binaire)
11    e. Vérification des ports suspects (ioc-persistent-connection)
12 5. Arrêt du conteneur
13 6. Collecte des logs avant suppression
14 7. Suppression du conteneur
15 8. Vérification CPU max (ioc-dynamic-core-limit)
16 9. Analyse des logs pour tentatives réseau suspectes (ioc-url)
17 10. Construction et retour de l'objet IOC_Dynamic

```

6.6 Module parser.py

parser.py permet l'intégration dans des pipelines CI/CD en extrayant automatiquement les images référencées dans des fichiers de configuration. Il expose un point d'entrée unique `extract_images` qui gère la détection du type de fichier et délègue à la fonction d'extraction appropriée.

6.6.1 Détection du type de fichier (`detect_file_type`)

Listing 18: Détection du type de fichier

```
1 def detect_file_type(filepath: str) -> str:
```

Détecte automatiquement si le fichier est un `docker-compose.yml` ou un `.gitlab-ci.yml` en inspectant les clés présentes dans le yaml. L'utilisateur n'a pas besoin de spécifier le type: la commande `scan-file` accepte n'importe quel nom de fichier.

La logique de détection est la suivante :

Listing 19: Logique de détection

```
1     if "services" in content:
2         return "docker-compose"
3     if "stages" in content or any(
4         isinstance(v, dict) and "script" in v
5         for v in content.values()
6         if isinstance(v, dict)
7     ):
8         return "gitlab-ci"
```

La présence de la clé `services` est caractéristique d'un `docker-compose.yml`. Pour le GitLab CI, on vérifie la présence de `stages` (déclaration explicite des stages) ou la présence de jobs contenant une clé `script` (structure minimale d'un job GitLab CI). Si aucune de ces conditions n'est satisfaite, la fonction retourne "unknown" et `extract_images` lève une exception.

6.6.2 Extraction docker-compose (`extract_images_docker_compose`)

Parcourt la section `services` du fichier et extrait le champ `image` de chaque service. Retourne une liste de dictionnaires `{image, source}` où `source` indique le nom du service d'origine (ex: `service:web`). Cette information permet de tracer quelle image provient de quel service dans le rapport final.

Seuls les services ayant un champ `image` explicite sont inclus: les services construits localement via `build` sont ignorés car ils ne font pas référence à une image de registre externe.

Exemple de résultat pour un `docker-compose` avec deux services :

Listing 20: Logique de détection

```
1 [
2     {"image": "nginx:latest", "source": "service:web"},
3     {"image": "fake-miner:latest", "source": "service:miner"}
4 ]
```


6.6.3 Extraction gitlab-ci (extract_images_gitlab_ci)

Extrait les images à deux niveaux : l'image globale définie à la racine du fichier (utilisée par défaut par tous les jobs) et les images définies individuellement par job. Les deux niveaux sont indépendants: un job peut surcharger l'image globale avec sa propre image.

La fonction gère les deux formats supportés par GitLab CI pour la déclaration d'une image :

Listing 21: Information d'extraction

```
1 # Format string simple
2 image: nginx:latest
3
4 # Format dictionnaire
5 image:
6   name: nginx:latest
7   entrypoint: [""]
```

Dans le second cas, `img.get("name")` extrait le nom de l'image depuis le dictionnaire. La logique `img if isinstance(img, str) else img.get("name")` gère les deux cas en une seule ligne.

Exemple de résultat pour un `.gitlab-ci.yml` avec image globale et deux jobs :

Listing 22: Logique de détection

```
1 [
2   {"image": "nginx:latest", "source": "global"},
3   {"image": "nginx:latest", "source": "job:build-job"},
4   {"image": "fake-miner:latest", "source": "job:test-job"}
5 ]
```

6.6.4 Point d'entrée (extract_images)

Listing 23: Point d'entrée de l'extraction

```
1 def extract_images(filepath: str) -> list:
```

Fonction principale appelée par `main.py`. Elle valide d'abord l'existence du fichier, puis délègue à `detect_file_type` et à la fonction d'extraction appropriée. Lève une `FileNotFoundError` si le fichier est introuvable et une `ValueError` si le type de fichier n'est pas reconnu: ces exceptions sont interceptées dans `main.py` pour afficher un message d'erreur propre à l'utilisateur.

6.7 Module static.py

L'analyse statique est composée de 2 phases. La première phase est de lire les différentes couches de l'image de conteneur et la deuxième phase à dresser la liste des fichiers présents dans un conteneur généré à partir de l'image suspecte.

6.7.1 collect_static_metrics

Cette fonction est le point d'entrée du module. Elle appelle les fonctions `ioc_rules_reader` pour obtenir un dictionnaire des indicateurs de compromission et `search_through_images_files` pour débiter l'analyse. Elle retourne un objet `IOC.Static` créé par la fonction `search_through_images_files`. Elle crée aussi un objet `DockerClient` qui sera envoyé à cette fonction.

Listing 24: collect_static_metric

```

1 def collect_static_metrics(image_name: str, flag_deep_search=False):
2     rules_yaml = ioc_rules_reader()
3     docker_client = docker.from_env()
4     results = search_through_image_files(docker_client, rules_yaml, image_name,
5                                         False)
6     return results

```

6.7.2 ioc_rules_reader

Listing 25: ioc_rules_reader

```

1 def ioc_rules_reader() -> dict:
2     print("[*] Lecture du fichier de regles")
3     current_dir = os.getcwd()
4     filepath = ""
5     if (os.path.exists(os.path.join(current_dir, "config/ioc_rules.yaml"))):
6         filepath = os.path.join(current_dir, 'config/ioc_rules.yaml')
7     else:
8         parent_dir = os.path.dirname(current_dir)
9         filepath = os.path.join(parent_dir, 'config/ioc_rules.yaml')
10
11     rules_yaml = {}
12     try:
13         with open(filepath, "r") as stream:
14             rules_yaml = yaml.safe_load(stream)
15             #print(rules_yaml)
16     except FileNotFoundError:
17         print("[*] Fichier de regle introuvable")
18
19     print("[*] Lecture du fichier de regles termine")
20     print(f"{'='*50}\n")
21     return rules_yaml

```

Le module commence par lire les règles IoC du fichier ioc_rules.yaml et retourne un dictionnaire.

6.7.3 search_through_image_files

Listing 26: search_through_image_files

```

1 def search_through_image_files(docker_client: DockerClient,
2     rules_yaml: dict, image_name: str, deep_search: bool):

```

Les règles utiles lors de cette phase seront celles correspondant à des commandes bash telles que `sh`, `curl`, et `wget` et à des noms de binaires associés au minage de cryptomonnaie telle que `xmrig` et `minerd`. Ces mots clés indiqueront si, lors de la construction de l'image, le créateur a inséré un mineur de crypto-monnaie.

```

1     print("[*] Extraction des indicateurs de compromission...")
2
3     ioc_rules_binaire = rules_yaml["rules"]["ioc-binaire"]["indicateur"]
4     ioc_rules_url = rules_yaml["rules"]["ioc-url"]["indicateur"]
5     ioc_rules_dockerfile = rules_yaml["rules"]["ioc-dockerfile"]["regex_patterns"]
6     ioc_rules_dockerfile_compile = {}
7     suspect_commands = {}
8     for ioc in ioc_rules_binaire:
9         suspect_commands[ioc] = []
10    for ioc in ioc_rules_url:

```

```

11     suspect_commands[ioc] = []
12     for ioc in ioc_rules_dockerfile:
13         ioc_rules_dockerfile_compile[ioc] = re.compile(ioc)
14         suspect_commands[ioc] = []
15     print("[*] Extraction termine")
16     print(f"{'='*50}\n")

```

La phase d'analyse des couches de l'image consiste à déterminer si chacune des couches contient correspondant à des commandes exécutant du code ou téléchargeant des fichiers binaires suspects vers une source. Dans la librairie python Docker Api, la classe `Image` a une fonction nommée `history()` qui retourne sous forme de liste de dictionnaires dont chacun d'entre ces dictionnaires correspond à une couche de l'image. Après extraire le champ `CreatedBy` de chacun des dictionnaires, les commandes ayant créé la couche apparaissent. L'outil analyse ensuite chacun des groupes de commande pour vérifier si les indicateurs sont présents. Ensuite, l'outil détermine les fichiers créés dans le conteneur pour la prochaine phase.

```

1     for layer in list_history:
2         for ioc in ioc_rules_url:
3             if (ioc in layer['CreatedBy']):
4                 suspect_commands[ioc].append(layer['CreatedBy'])
5         for ioc in ioc_rules_binaire:
6             if (ioc in layer['CreatedBy']):
7                 suspect_commands[ioc].append(layer['CreatedBy'])
8         for ioc in ioc_rules_dockerfile:
9             if (ioc_rules_dockerfile_compile[ioc]
10                 .search(layer['CreatedBy']) != None):
11                 layer_composition = layer['CreatedBy'].split(" ")
12                 for composant in layer_composition:
13                     if (ioc_rules_dockerfile_compile[ioc]
14                         .search(composant) != None):
15                         suspect_commands[ioc].append(composant)
16
17
18     crossreferenced_commands = []
19     for ioc_url in ioc_rules_url:
20         for sus_url_command in suspect_commands[ioc_url]:
21             for ioc_bin in ioc_rules_binaire:
22                 if (any(xmrig_command == sus_url_command for
23                     xmrig_command in suspect_commands[ioc_bin])):
24                     crossreferenced_commands.append(sus_url_command)
25
26     #Capture les urls malveillantes
27     suspect_urls = []
28     for command in crossreferenced_commands:
29         if ("curl" in command or "wget" in command):
30             splited_command = command.split(" ")
31             for argument in splited_command:
32                 if ("http" in argument or ".com" in
33                     argument or ".net" in argument):
34                     if (argument not in suspect_urls):
35                         suspect_urls.append(argument)

```

La phase d'analyse du système de fichier du conteneur consiste à valider la présence des fichiers suspects que la première phase a trouvé. Docker permet de créer des conteneurs sans les démarrer, empêchant l'image d'enclencher les binaires malicieux. Ensuite, l'outil roule la commande `export` de Docker pour exporter le système de fichiers du conteneur dans un fichier d'archive, puis la commande `tar -t > fichier.txt` pour obtenir la liste de tous les fichiers dans le conteneur dans un document texte. L'outil termine en confirmant la présence des fichiers trouvés lors de la phase précédente. À la fin, le conteneur de l'image est détruit.

```

1 #Debut l'analyse du systeme de fichiers
2 print("[*] Analyse du systeme de fichiers du conteneur.")
3 print("[*] Creation du conteneur...")
4 sus_container = docker_client.containers.create(image_name,
5     detach=True, network="cryptojack-timeout")
6 container_name = image_name.replace("/", "_")
7 print("[*] Dressage de la liste des fichiers...")
8 os.system(f"docker export {sus_container.name}
9 | tar -t > {container_name}_list_of_files.txt")
10
11 print("[*] Analyse du systeme de fichiers...")
12 sus_files_created_in_sus_container = []
13
14 #Cette boucle cherche a travers le systeme de fichier
15 pour trouver les fichiers suspects dans les commandes suspected
16 try:
17     for command in crossreferenced_commands:
18         splited_command = command.split(" ")
19         for argument in splited_command:
20             if "/etc/" in argument or "/usr/bin/" in argument:
21                 with open(f"{container_name}_list_of_files.txt",
22                     "r") as file:
23                     for line in file:
24                         if (argument[1:] == line.strip()):
25                             if (argument not in
26                                 sus_files_created_in_sus_container):
27                                 sus_files_created_in_sus_container
28                                     .append(argument)
29 except FileNotFoundError:
30     print(f"[*] ERREUR: Le fichier {container_name}_list_of_files.txt
31         n'existe pas!")
32 print("[*] Analyse du systeme de fichiers termine")

```

6.8 Module report.py et interface.py

6.8.1 report.py

Ce script python contient la fonction `convert_to_json` qui utilise les données sous forme de classe de `interface.py` pour les convertir en fichier JSON, utilisable pour des logs et l'affichage d'informations. Une fonction `write_json` est fournie pour donner la possibilité de diriger où sera situé le fichier JSON créé.

6.8.2 interface.py

Le fichier `interface.py` contient 2 classes représentant les données générées par l'analyse statique (`IOC_Static`) et l'analyse dynamique (`IOC.Dynamic`).

Listing 27: `interface.py`

```

1 class IOC_Static:
2     image_name: str = ""
3     xmrig_detected: bool = False
4     minerd_detected: bool = False
5     kminer_detected: bool = False
6     sh_detected: bool = False
7     sh_samples: int = 0
8     curl_detected: bool = False
9     curl_samples: int = 0
10    wget_detected: bool = False

```

```

11     wget_samples: int = 0
12     higher_permission_detected: bool = False
13     malicious_urls: list = field(default_factory=list)
14     malicious_files: list = field(default_factory=list)
15     malicious_pool: list = field(default_factory=list)
16     malicious_stratum: list = field(default_factory=list)
17
18
19 class IOC_Dynamic:
20     image_name: str = ""
21     runtime_sec: int = 0
22     cpu_samples: int = 0
23     mem_samples: int = 0
24     cpu_avg: float = 0.0
25     cpu_max: float = 0.0
26     mem_avg: float = 0.0
27     mem_max: float = 0.0
28     list_process: list = field(default_factory=list)
29     connections: list = field(default_factory=list)
30     ioc_hits: list = field(default_factory=list)
31     logs: list = field(default_factory=list)

```

Les données statiques comprennent le nom de l'image et différents indices de système compromis qui peut être détecté comme `xmrig_detected` qui indique si un texte contient du xmrig. Les données d'analyse dynamique contiennent des données similaires en lien avec les indices de système compromis en plus du temps de d'exécution de l'image.

6.9 Fichiers de test

6.9.1 fake-miner: Image malveillante simulée

L'image `fake-miner` simule les comportements caractéristiques d'un cryptomineur sans utiliser de vrai malware. Elle a été conçue pour déclencher les IoC suivants :

textbfIndicateurs dynamique:

- CPU soutenu à 100% via `dd if=/dev/urandom` et calcul Python intensif
- Processus `kworker`: nom obfusqué imitant un processus kernel Linux légitime
- Processus `python3 /gpu_sim.py` simulant une charge GPU via calcul mathématique intensif
- Tentatives de connexion vers les ports 3333 et 4444 via `netcat`
- Tentative de téléchargement via `curl` vers un pool de minage connu

Indicateurs statiques:

- URL de pool de minage `stratum+tcp://pool.minexmr.com:4444` dans les scripts
- Variable d'environnement `POOL_URL` exposant la configuration du mineur
- Variable d'environnement `WALLET_ADDRESS` avec une adresse crypto

Le choix du nom `kworker` est délibéré : il illustre la technique d'obfuscation par imitation de processus système. Sans la règle `kworker` dans `ioc_rules.yaml`, le binaire ne serait détecté que par son comportement CPU, démontrant la valeur de l'approche combinée.

6.9.2 docker-compose.yml de test

Listing 28: docker-compose de test

```
1 services:
2   web:
3     image: nginx:latest
4   miner:
5     image: fake-miner:latest
```

Ce fichier permet de tester la commande `scan-file` avec deux images aux comportements opposés : une image légitime (nginx) et une image malveillante (fake-miner). L'analyse en parallèle permet de vérifier que les métriques ne se mélangent pas entre les deux conteneurs.

6.9.3 .gitlab-ci.yml de test

Listing 29: gitlab ci de test

```
1 stages:
2   - build
3   - test
4
5 image: nginx:latest
6
7 build-job:
8   stage: build
9   image: nginx:latest
10  script:
11    - echo "Build stage"
12
13 test-job:
14   stage: test
15   image: fake-miner:latest
16   script:
17     - echo "Test stage"
```

Ce fichier teste l'extraction d'images depuis un fichier GitLab CI avec une image globale et des images par job. Il permet de valider que le parser gère correctement les deux niveaux de déclaration d'images.

6.9.4 run_tests.sh

Le script est un outil de tests automatisés pour valider que l'outil de détection fonctionne correctement. Voici une explication complète section par section.

Vue d'ensemble

`run_tests.sh` est un script Bash de tests automatisés qui valide le bon fonctionnement de l'outil sur deux axes : la détection sur des images connues (légitime et malveillante) et le bon fonctionnement du pipeline d'analyse par fichier yaml. Il est conçu pour être personnalisable. Un nouveau contributeur peut ajouter ses propres tests sans modifier la structure existante.

Utilisation

Listing 30: Commandes de `run_tests.sh`

```
1 chmod +x tests/run_tests.sh
2
3 # Tous les tests
```

```

4 ./tests/run_tests.sh
5
6 # Tests sur les images seulement
7 ./tests/run_tests.sh images
8
9 # Tests pipeline seulement
10 ./tests/run_tests.sh pipeline

```

Configuration

Listing 31: Configuration run_tests.sh

```

1 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
2 PROJECT_DIR="$(dirname "$SCRIPT_DIR")"
3 DURATION=15
4 INTERVAL=2
5 OUTPUT_DIR="$PROJECT_DIR/output/tests"
6 PASS=0
7 FAIL=0

```

Le script détermine automatiquement son emplacement sur le système de fichiers (SCRIPT_DIR) et remonte d'un niveau pour trouver la racine du projet (PROJECT_DIR). Cela permet de lancer le script depuis n'importe quel répertoire sans chemin absolu. DURATION et INTERVAL configurent la durée et la fréquence des analyses dynamiques. PASS et FAIL sont des compteurs incrémentés au fil des tests pour le résumé final.

Fonctions utilitaires (helpers)

Ces fonctions sont réutilisables par tous les tests, y compris les tests personnalisés. **build_image** :

Listing 32: build_image

```

1 build_image() {
2     docker build -t "$tag" "$path" -q
3 }

```

Construit une image Docker depuis un Dockerfile local. L'option -q (quiet) supprime la sortie de build pour ne pas polluer la console pendant les tests. **run_analyze** :

Listing 33: run_analyze

```

1 run_analyze() {
2     python main.py analyze \
3         --image "$image" \
4         --duration $DURATION \
5         --interval $INTERVAL \
6         --output "$output" > /dev/null 2>&1
7 }

```

Lance l'analyse complète (statique + dynamique) via la commande CLI. La redirection '`/dev/null 2>&1`' supprime toute la sortie stdout et stderr: les résultats sont lus directement depuis les rapports JSON générés. **get_ioc_counts** :

Listing 34: get_ioc_counts

```

1 get_ioc_count() {
2     python3 -c "import json; d=json.load(open('$1')); print(len(d['ioc_hits']))"
3 }

```

Lit le rapport JSON dynamique et retourne le nombre d'IOC hits. Utilisé pour vérifier si une image a déclenché des alertes sans analyser manuellement le fichier. **get_cpu_max** :

Listing 35: get_cpu_max

```
1 get_cpu_max() {
2     python3 -c "import json; d=json.load(open('$1')); print(d['cpu_max'])"
3 }
```

Extrait le CPU maximum observé depuis un rapport JSON dynamique. Utilisé pour valider les seuils de détection comportementale. **ioc_contains** :

Listing 36: ioc_contains

```
1 ioc_contains() {
2     python3 -c "
3         import json
4         d = json.load(open('$1'))
5         print('found' if any('$2' in h for h in d['ioc_hits']) else 'missing')
6     "
7 }
```

Vérifie si un mot clé précis est présent dans la liste des IOC hits. Retourne found ou missing pour simplifier les conditions bash. Par exemple, ioc_contains "\$REPORT" "kworker" vérifie si le binaire kworker a été détecté. **assert_report_exists** :

Listing 37: assert_report_exists

```
1 assert_report_exists() {
2     if [ ! -f "$report" ]; then
3         log_fail "$name - rapport JSON non généré : $report"
4         return 1
5     fi
6     return 0
7 }
```

Vérifie que le fichier JSON a bien été généré avant d'essayer de le lire. Si le fichier est absent, enregistre un échec et interrompt le test avec 'return 1' pour éviter des erreurs en cascade lors de la lecture du fichier. **setup** :

Listing 38: setup

```
1 setup() {
2     python main.py setup
3     mkdir -p "$OUTPUT_DIR"
4     build_image "fake-miner:latest" "tests/test_images/fake_miner"
5 }
```

Exécuté en premier avant tout test. Configure le réseau Docker isolé cryptojack-timeout via python main.py setup, crée le dossier de sortie des rapports de test, et construit l'image fake-miner depuis le Dockerfile local. Cela garantit que l'environnement complet est prêt avant le premier test.

Tests sur les images

test_nginx - validation image légitime :

Listing 39: Test nginx

```

1 test_nginx() {
2     run_analyze "nginx:latest" "$OUTPUT_DIR"
3     REPORT="$OUTPUT_DIR/nginx_latest_dynamic.json"
4     assert_report_exists "$REPORT" "nginx" || return
5
6     IOC_HITS=$(get_ioc_count "$REPORT")
7     CPU_MAX=$(get_cpu_max "$REPORT")
8     ...
9 }

```

Valide le comportement d'une image légitime. Deux conditions sont vérifiées : zéro IoC hit (absence de fausse alerte) et CPU max inférieur à 70% (comportement serveur web idle). Si l'une ou l'autre échoue, c'est un faux positif de l'outil.

test_fake_miner - validation image malveillante :

Listing 40: Test fake-miner

```

1 test_fake_miner() {
2     run_analyze "fake-miner:latest" "$OUTPUT_DIR"
3     REPORT="$OUTPUT_DIR/fake-miner_latest_dynamic.json"
4     assert_report_exists "$REPORT" "fake-miner" || return
5
6     IOC_HITS=$(get_ioc_count "$REPORT")
7     CPU_MAX=$(get_cpu_max "$REPORT")
8     KWORKER=$(ioc_contains "$REPORT" "kworker")
9     ...
10 }

```

Valide la détection d'une image malveillante simulée. Trois conditions sont vérifiées : au moins un IoC hit, CPU max supérieur ou égal à 70%, et présence spécifique du binaire kworker dans les IoC hits. Ce test valide à la fois la détection comportementale (CPU) et la détection par nom de binaire.

Tests pipeline

test_pipeline_docker_compose :

Lance scan-file sur le fichier tests/test_images/docker-compose.yml contenant deux services (nginx et fake-miner) et vérifie que les deux rapports JSON ont été générés. Valide que le parser extrait correctement les images et que l'analyse parallèle fonctionne. **test_pipeline_gitlab_ci :**

Lance scan-file sur le fichier tests/test_images/.gitlab-ci.yml et vérifie que la commande s'exécute sans erreur. Valide que le parser détecte correctement le format GitLab CI avec image globale et images par job.

Tests personnalisés

Le script inclut un patron commenté pour ajouter de nouveaux tests sans modifier la structure existante :

Listing 41: Patron tests personnalisés

```

1 # test\_mon\_image() {
2 #     build\_image "mon-image:latest" "tests/test\_images/mon\_image"
3 #     run\_analyze "mon-image:latest" "\$OUTPUT\_DIR"
4 #     REPORT="\$OUTPUT\_DIR/mon-image\_latest\_dynamic.json"
5 #     assert\_report\_exists "\$REPORT" "mon-image" || return
6 #
7 #     IOC\_HITS=\$(get\_ioc\_count "\$REPORT")
8 #     BINAIRE=\$(ioc\_contains "\$REPORT" "nom-du-binaire")
9 #     CPU\_MAX=\$(get\_cpu\_max "\$REPORT")
10 #     ...
11 # }

```

Pour activer un test personnalisé, il suffit de décommenter la fonction et d'ajouter son appel dans le bloc images du point d'entrée.

Cleanup

Listing 42: cleanup

```
1 cleanup() {
2     python main.py cleanup --image fake-miner:latest 2>/dev/null || true
3 }
```

Supprime l'image fake-miner après les tests pour libérer l'espace disque. Le `— true` empêche le script d'échouer si l'image a déjà été supprimée. Le `'2>/dev/null'` masque les messages d'erreur non critiques.

Résumé et code de sortie

```
1 print_summary() {
2     echo -e " ${GREEN}PASS${NC} : $PASS"
3     echo -e " ${RED}FAIL${NC} : $FAIL"
4
5     if [ "$FAIL" -gt 0 ]; then
6         exit 1
7     fi
8 }
```

Affiche le nombre de PASS et FAIL avec un code couleur. Si au moins un test a échoué, le script retourne un code de sortie non-zéro (exit 1). Ce comportement est essentiel pour l'intégration CI/CD: un pipeline GitLab CI ou GitHub Actions détecte automatiquement ce code et bloque le déploiement si des tests échouent.

Résultats CLI

```
1 (venv) lch@cryptojacking:~/cryptojack-detector/pj-recherche-26h-6$ ./tests/
   run_tests.sh
2 [INFO] Configuration de l'environnement...
3 cryptojack-timeout already exists!
4 /etc/docker already exists.
5 /etc/docker/daemon.json already exists.
6 [INFO] Construction de l'image fake-miner:latest...
7 sha256:3c6cc53af0a3b463f84b06cf9344c074886514bfec02a0b19807533e66a4d72b
8 [INFO] Setup terminé.
9
10 === Tests sur les images ===
11 [INFO] Test - nginx:latest (image légitime, aucun IoC attendu)
12 [PASS] nginx - aucun IoC détecté (attendu)
13 [PASS] nginx - CPU max à 6.37% (sous le seuil de 70%)
14 [INFO] Test - fake-miner:latest (image malveillante simulée)
15 [PASS] fake-miner - 2 IoC détecté(s) (attendu)
16 [PASS] fake-miner - CPU max à 100.0% (dépassé le seuil de 70%)
17 [PASS] fake-miner - binaire kworker détecté
18
19
20 === Tests pipeline ===
21 [INFO] Test pipeline - docker-compose.yml
22 [PASS] docker-compose - les deux rapports JSON générés
23 [INFO] Test pipeline - .gitlab-ci.yml
24 [PASS] gitlab-ci - scan-file exécuté sans erreur
```

```

25 [INFO] Nettoyage des images de test...
26 [+] Image fake-miner:latest supprimée.
27 [INFO] Nettoyage terminé.
28
29 =====
30     Résumé des tests
31 =====
32     PASS : 7
33     FAIL : 0
34 =====

```

6.10 Intégration dans un pipeline CI/CD

L'outil est conçu pour s'intégrer comme première étape de validation dans un pipeline CI/CD, avant le déploiement des conteneurs en production. L'objectif est de détecter les images malveillantes le plus tôt possible dans le cycle de développement, avant qu'elles n'atteignent un environnement de production.

6.10.1 Workflow typique

Listing 43: Exemple workflow intégration CI/CD

```

1 [Push du code]
2     \rightarrow [setup: configuration de environnement isolé]
3     \rightarrow [scan-file: extraction des images depuis docker-compose / gitlab-ci
4         ]
5     \rightarrow [analyze: analyse dynamique de chaque image]
6     \rightarrow [analyze: analyse statique de chaque image]
7     \rightarrow [scoring: corrélation des IoC statiques et dynamiques] (à venir)
8     \rightarrow [Rapport JSON généré pour chaque image]
9     \rightarrow [Décision : déployer ou bloquer selon le score]

```

6.10.2 Prérequis pour l'intégration

Pour intégrer l'outil dans un pipeline, le runner CI/CD doit satisfaire les conditions suivantes :

- Docker installé et accessible depuis le runner.
- Python 3.10+ avec les dépendances installées (pip install -r requirements.txt).
- Permissions suffisantes pour créer des réseaux Docker et lancer des conteneurs
- Le réseau cryptojack-timeout créé via python 'main.py setup' avant la première exécution.

6.10.3 Exemple d'intégration GitLab CI

Listing 44: Exemple intégration gitlab CI

```

1 stages:
2   - build
3   - security
4   - deploy
5
6 security-scan:
7   stage: security
8   before_script:
9     - pip install -r requirements.txt
10    - python main.py setup

```

```

11 script:
12   # Analyser toutes les images déclarées dans le docker-compose
13   - python main.py scan-file --file docker-compose.yml
14   --duration 60 --output ./security-reports
15   # Optionnel : analyser une image spécifique
16   # - python main.py analyze --image mon-image:latest --duration 60
17 after_script:
18   # Nettoyage des images analysées
19   - python main.py cleanup --image mon-image:latest || true
20 artifacts:
21   paths:
22     - security-reports/
23   expire_in: 7 days
24 rules:
25   - if: $CI_PIPELINE_SOURCE == "merge_request_event"
26   - if: $CI_COMMIT_BRANCH == "main"

```

Le stage `security` est placé après `build` et avant `deploy`: si l'analyse détecte des IoC, le pipeline peut être configuré pour bloquer le déploiement. Les rapports JSON sont archivés comme artefacts GitLab pour consultation ultérieure.

6.10.4 Exemple d'intégration GitHub Actions

Listing 45: Exemple intégration GitHub Actions

```

1 name: Security Scan
2
3 on:
4   push:
5     branches: [main]
6   pull_request:
7     branches: [main]
8
9 jobs:
10  security-scan:
11    runs-on: ubuntu-latest
12    steps:
13      - uses: actions/checkout@v3
14
15      - name: Setup Python
16        uses: actions/setup-python@v4
17        with:
18          python-version: '3.11'
19
20      - name: Install dependencies
21        run: pip install -r requirements.txt
22
23      - name: Setup isolated network
24        run: python main.py setup
25
26      - name: Scan images
27        run: |
28          python main.py scan-file \
29            --file docker-compose.yml \
30            --duration 60 \
31            --output ./security-reports
32
33      - name: Upload reports
34        uses: actions/upload-artifact@v3

```

```

35     with:
36         name: security-reports
37         path: security-reports/
38         retention-days: 7
39
40     - name: Cleanup
41       if: always()
42       run: python main.py cleanup --image mon-image:latest || true

```

6.10.5 Avantages de l'approche par fichier yaml

La commande `scan-file` est la clé de l'intégration CI/CD pour deux raisons principales.

Premièrement, elle ne requiert aucune modification du pipeline existant: elle lit les fichiers de configuration déjà présents dans le projet (`docker-compose.yml`, `.gitlab-ci.yml`) sans que l'équipe ait à lister manuellement les images à analyser. Toute nouvelle image ajoutée dans ces fichiers sera automatiquement incluse dans l'analyse.

Deuxièmement, l'analyse en parallèle des images réduit le temps total d'exécution proportionnellement au nombre d'images. Un projet avec 4 images prend autant de temps qu'un projet avec 1 image, ce qui est important dans un contexte de pipeline où chaque minute compte.

6.10.6 Limites dans un contexte CI/CD

Durée d'observation: Une durée de 60 secondes est recommandée en production pour donner le temps aux comportements malveillants de se manifester. Des durées plus courtes (15-30s) conviennent pour les tests rapides mais peuvent manquer des malwares à démarrage différé.

Runner dédié: Il est fortement recommandé d'utiliser un runner CI/CD dédié exclusivement à l'analyse de sécurité, distinct du runner de build. Un malware qui s'échappe du conteneur ne devrait pas atteindre l'environnement de build.

Pas de blocage automatique: Dans l'état actuel du prototype, l'outil génère un rapport JSON mais ne bloque pas automatiquement le pipeline selon le score. L'intégration d'un mécanisme de décision automatique (exit code non-zéro si IoC détectés) est une perspective d'amélioration.

6.11 Résultats du projet académique

6.11.1 Résultats de l'analyse dynamique

Listing 46: Rapport JSON `fake-miner_latest_dynamic.json`

```

1  {
2    "analysis_type": "Dynamic",
3    "image_name": "fake-miner:latest",
4    "runtime_sec": 15,
5    "cpu_samples": 5,
6    "mem_samples": 5,
7    "cpu_avg": 100.0,
8    "cpu_max": 100.0,
9    "mem_avg": 23.28,
10   "mem_max": 24.29,
11   "list_process": [
12     "/bin/sh /usr/bin/kworker",
13     "python3 /gpu_sim.py",
14     "/bin/sh /entrypoint.sh",

```

```

15         "dd if /dev/urandom of /dev/null bs 1M count 100"
16     ],
17     "connections": [
18         "eth0: rx=2944B tx=358B"
19     ],
20     "ioc_hits": [
21         "[ioc-cpu] CPU max \u00e0 100.00% d\u00e9passe le seuil de 70%",
22         "[ioc-binaire] Processus suspects: ['kworker']"
23     ],
24     "logs": [
25         "100+0 records in",
26         "100+0 records out",
27         "100+0 records in",
28         "100+0 records out",
29         "104857600 bytes (100.0MB) copied, 0.633591 seconds, 157.8MB/s",
30         "104857600 bytes (100.0MB) copied, 0.639159 seconds, 156.5MB/s",
31         "100+0 records in",
32         "100+0 records out",
33         "100+0 records in",
34         "100+0 records out",
35         "104857600 bytes (100.0MB) copied, 0.773503 seconds, 129.3MB/s",
36         "104857600 bytes (100.0MB) copied, 0.747246 seconds, 133.8MB/s",
37         "100+0 records in",
38         "100+0 records out",
39         "100+0 records in",
40         "100+0 records out",
41         "104857600 bytes (100.0MB) copied, 0.681567 seconds, 146.7MB/s",
42         "104857600 bytes (100.0MB) copied, 0.660994 seconds, 151.3MB/s",
43         "100+0 records in",
44         "100+0 records out",
45         "104857600 bytes (100.0MB) copied, 0.643130 seconds, 155.5MB/s",
46         "100+0 records in",
47         "100+0 records out",
48         "104857600 bytes (100.0MB) copied, 0.731574 seconds, 136.7MB/s",
49         "100+0 records in",
50         "100+0 records out",
51         "104857600 bytes (100.0MB) copied, 0.683900 seconds, 146.2MB/s",
52         "100+0 records in",
53         "100+0 records out",
54         "104857600 bytes (100.0MB) copied, 0.724018 seconds, 138.1MB/s",
55         "100+0 records in",
56         ...
57     ]
58 }

```

Listing 47: Retour CLI fake-miner:latest

```

1  =====
2  Analyse de l'image : fake-miner:latest
3  Durée           : 15s
4  Intervalle      : 2s
5  =====
6
7  [*] Vérification de l'image fake-miner:latest...
8  [*] Image trouvée localement, pas de pull nécessaire.
9  [*] Lancement du conteneur...
10 [*] Collecte des métriques pendant 15 secondes...
11 CPU: 100.00% MEM: 23.64 MB PROC: ['/bin/sh /entrypoint.sh',
12   '/bin/sh /usr/bin/kworker', 'python3 /gpu_sim.py',

```

```

13  '/bin/sh /usr/bin/kworker',
14  'dd if /dev/urandom of /dev/null bs 1M count 100',
15  'dd if /dev/urandom of /dev/null bs 1M count 100']
16  NET: ['eth0: rx=836B tx=274B']
17  CPU: 100.00% MEM: 23.98 MB PROC: ['/bin/sh /entrypoint.sh',
18  '/bin/sh /usr/bin/kworker', 'python3 /gpu_sim.py',
19  '/bin/sh /usr/bin/kworker',
20  'dd if /dev/urandom of /dev/null bs 1M count 100',
21  'dd if /dev/urandom of /dev/null bs 1M count 100']
22  NET: ['eth0: rx=2411B tx=316B']
23  CPU: 100.00% MEM: 23.97 MB PROC: ['/bin/sh /entrypoint.sh',
24  '/bin/sh /usr/bin/kworker', 'python3 /gpu_sim.py',
25  '/bin/sh /usr/bin/kworker',
26  'dd if /dev/urandom of /dev/null bs 1M count 100',
27  'dd if /dev/urandom of /dev/null bs 1M count 100']
28  NET: ['eth0: rx=2837B tx=358B']
29  CPU: 100.00% MEM: 23.84 MB PROC: ['/bin/sh /entrypoint.sh',
30  '/bin/sh /usr/bin/kworker', 'python3 /gpu_sim.py',
31  '/bin/sh /usr/bin/kworker',
32  'dd if /dev/urandom of /dev/null bs 1M count 100',
33  'dd if /dev/urandom of /dev/null bs 1M count 100']
34  NET: ['eth0: rx=2944B tx=358B']
35  CPU: 100.00% MEM: 23.13 MB PROC: ['/bin/sh /entrypoint.sh',
36  '/bin/sh /usr/bin/kworker', 'python3 /gpu_sim.py',
37  '/bin/sh /usr/bin/kworker',
38  'dd if /dev/urandom of /dev/null bs 1M count 100',
39  'dd if /dev/urandom of /dev/null bs 1M count 100']
40  NET: ['eth0: rx=2944B tx=358B']
41  [*] Arrêt du conteneur...
42  [*] Suppression du conteneur...
43  Resulats Analyse dynamique
44  =====
45  CPU moyen : 100.0%
46  CPU max : 100.0%
47  MEM moy : 23.71 MB
48  MEM max : 23.98 MB
49  Processus : ['/bin/sh /usr/bin/kworker', '/bin/sh /entrypoint.sh',
50  'dd if /dev/urandom of /dev/null bs 1M count 100', 'python3 /gpu_sim.py']
51  Connexions: ['eth0: rx=2944B tx=358B']
52  IoC hits : [['ioc-cpu] CPU max à 100.00% dépasse le seuil de 70%',
53  "[ioc-binaire] Processus suspects: ['kworker']"]
54  Logs suspects: ['100+0 records in', '100+0 records out',
55  '104857600 bytes (100.0MB) copied, 0.594339 seconds, 168.3MB/s',
56  '100+0 records in', '100+0 records out']
57  =====
58
59  [*] Rapport JSON genere pour l'analyse dynamique :
60  ./output/fake-miner_latest_dynamic.json

```

Listing 48: Rapport JSON nginx_latest_dynamic.json

```

1  {
2      "analysis_type": "Dynamic",
3      "image_name": "nginx:latest",
4      "runtime_sec": 15,
5      "cpu_samples": 5,
6      "mem_samples": 5,
7      "cpu_avg": 0.92,
8      "cpu_max": 4.58,

```

```

9      "mem_avg": 2.65,
10     "mem_max": 2.65,
11     "list_process": [
12         "nginx: worker process",
13         "nginx: master process nginx -g daemon off;"
14     ],
15     "connections": [
16         "eth0: rx=2822B tx=126B"
17     ],
18     "ioc_hits": [],
19     "logs": [
20         "/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty,
21             will attempt to perform configuration",
22         "/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/"
23         ,
24         "/docker-entrypoint.sh: Launching /docker-entrypoint.d/
25             10-listen-on-ipv6-by-default.sh",
26         "10-listen-on-ipv6-by-default.sh: info: Getting the checksum of
27             /etc/nginx/conf.d/default.conf",
28         "10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in
29             /etc/nginx/conf.d/default.conf",
30         "/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/
31             15-local-resolvers.envsh",
32         "/docker-entrypoint.sh: Launching /docker-entrypoint.d/
33             20-envsubst-on-templates.sh",
34         "/docker-entrypoint.sh: Launching /docker-entrypoint.d/
35             30-tune-worker-processes.sh",
36         "/docker-entrypoint.sh: Configuration complete; ready for start up",
37         "2026/03/30 19:01:03 [notice] 1#1: using the \"epoll\" event method",
38         "2026/03/30 19:01:03 [notice] 1#1: nginx/1.29.6",
39         "2026/03/30 19:01:03 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19) "
40         ,
41         "2026/03/30 19:01:03 [notice] 1#1: OS: Linux 6.12.73+deb13-amd64",
42         "2026/03/30 19:01:03 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288",
43         "2026/03/30 19:01:03 [notice] 1#1: start worker processes",
44         "2026/03/30 19:01:03 [notice] 1#1: start worker process 29",
45         "2026/03/30 19:01:18 [notice] 1#1: signal 3 (SIGQUIT) received, shutting
46             down",
47         "2026/03/30 19:01:18 [notice] 29#29: gracefully shutting down",
48         "2026/03/30 19:01:18 [notice] 29#29: exiting",
49         "2026/03/30 19:01:18 [notice] 29#29: exit",
50         "2026/03/30 19:01:18 [notice] 1#1: signal 17 (SIGCHLD) received from 29",
51         "2026/03/30 19:01:18 [notice] 1#1: worker process 29 exited with code 0",
52         "2026/03/30 19:01:18 [notice] 1#1: exit"
53     ]
54 }

```

Listing 49: Retour CLI nginx:latest

```

1  =====
2  Analyse de l'image : nginx:latest
3  Durée           : 15s
4  Intervalle      : 2s
5  =====
6
7  [*] Vérification de l'image nginx:latest...
8  [*] Image trouvée localement, pas de pull nécessaire.
9  [*] Lancement du conteneur...
10 [*] Collecte des métriques pendant 15 secondes...

```



```

11 CPU: 16.45% MEM: 7.62 MB PROC: ['nginx: master process nginx -g daemon off;',
12 'nginx: worker process'] NET: ['eth0: rx=526B tx=84B']
13 CPU: 0.00% MEM: 7.61 MB PROC: ['nginx: master process nginx -g daemon off;',
14 'nginx: worker process'] NET: ['eth0: rx=2261B tx=126B']
15 CPU: 0.00% MEM: 7.61 MB PROC: ['nginx: master process nginx -g daemon off;',
16 'nginx: worker process'] NET: ['eth0: rx=2645B tx=126B']
17 CPU: 0.00% MEM: 7.61 MB PROC: ['nginx: master process nginx -g daemon off;',
18 'nginx: worker process'] NET: ['eth0: rx=2752B tx=126B']
19 CPU: 0.00% MEM: 7.61 MB PROC: ['nginx: master process nginx -g daemon off;',
20 'nginx: worker process'] NET: ['eth0: rx=2822B tx=126B']
21 [*] Arrêt du conteneur...
22 [*] Suppression du conteneur...
23 Resultats Analyse dynamique
24 =====
25 CPU moyen : 3.29%
26 CPU max : 16.45%
27 MEM moy : 7.61 MB
28 MEM max : 7.62 MB
29 Processus : ['nginx: master process nginx -g daemon off;', 'nginx: worker process
']
30 Connexions: ['eth0: rx=2822B tx=126B']
31 IoC hits : Aucun
32 Logs suspects: ['/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty,
33 will attempt to perform configuration', '/docker-entrypoint.sh:
34 Looking for shell scripts in /docker-entrypoint.d/',
35 '/docker-entrypoint.sh: Launching
36 /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh',
37 '10-listen-on-ipv6-by-default.sh: info: Getting the checksum of
38 /etc/nginx/conf.d/default.conf',
39 '10-listen-on-ipv6-by-default.sh:
40 info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf']
41 =====
42
43 [*] Rapport JSON genere pour l'analyse dynamique : ./output/nginx_latest_dynamic.
json

```

Méthodologie d'évaluation

Les tests ont été conduits sur deux images représentant des cas opposés : **nginx:latest** comme image de référence légitime et **fake-miner:latest** comme image malveillante simulée. Chaque analyse a été exécutée avec une durée d'observation de 15 secondes et un intervalle de collecte de 2 secondes, produisant 5 échantillons par métrique.

L'environnement d'exécution est une VM VirtualBox dédiée avec le réseau Docker isolé **cryptojack-timeout** configuré avec l'option **--internal** bloquant tout accès internet depuis les conteneurs.

Résultats comparatifs

Métrique	nginx:latest	fake-miner:latest
CPU moyen	3.29%	100.00%
CPU max	16.45%	100.00%
Mémoire moyenne	7.61MB	23.71MB
Mémoire max	7.62MB	23.98MB
Nombre de processus	2	4
IoC hits	0	2
Verdict	Légitime	Malveillant

Table 4: Résultats nginx et fake-miner

Analyse de l'image légitime - `nginx:latest`

L'analyse de `nginx:latest` produit un profil comportemental cohérent avec une image de serveur web en attente de requêtes. Le CPU présente un pic initial de 16.45% correspondant à la phase d'initialisation du serveur, suivi d'une utilisation nulle pendant les 4 mesures suivantes, reflétant un conteneur idle sans requêtes entrantes. La mémoire reste stable à 7.61 MB sur toute la durée d'observation.

Deux processus sont actifs : le processus maître `nginx` et un `worker process`, ce qui correspond exactement à la configuration par défaut de `nginx`. Les connexions réseau montrent un trafic entrant modeste (2 822 B reçus) correspondant aux health checks Docker, et un trafic sortant minimal (126 B), sans aucune connexion vers des ports suspects. L'analyse ne produit aucun IoC hit, confirmant que l'image est correctement classifiée comme légitime.

Les logs capturés reflètent uniquement le cycle d'initialisation normal de `nginx`, avec les scripts de configuration de l'entrypoint officiels.

Analyse de l'image malveillante simulée - `fake-miner:latest`

L'analyse de `fake-miner:latest` révèle un profil comportemental caractéristique d'une activité de cryptomining. Le CPU atteint et maintient 100% à chaque mesure sur l'ensemble des 15 secondes d'observation, sans aucune variation — ce comportement soutenu et constant est l'un des indicateurs les plus fiables d'un processus de minage actif.

Quatre processus distincts sont identifiés lors de l'exécution. Le processus `/bin/sh /entrypoint.sh` est le script d'orchestration principal. Le processus `/bin/sh /usr/bin/kworker`, présent en deux instances simultanées, est le faux mineur dont le nom imite délibérément un processus kernel Linux légitime (`kworker`) pour passer inaperçu. Le processus `python3 /gpu.sim.py` simule une charge GPU via des calculs mathématiques intensifs. Enfin, `dd if=/dev/urandom` génère une charge CPU soutenue via des opérations d'entrée/sortie en boucle, visible dans les logs sous forme de lignes répétitives `104857600 bytes (100.0MB) copied`.

La mémoire s'établit à 23.71 MB en moyenne, soit environ trois fois plus que `nginx`, reflétant l'empreinte mémoire des processus de calcul concurrents.

L'analyse produit deux IoC hits distincts. Le premier, `[ioc-cpu]`, est déclenché par le CPU max à 100% dépassant le seuil configuré de 70%. Le second, `[ioc-binaire]`, est déclenché par la détection du binaire `kworker` dans la liste des processus actifs: binaire répertorié dans `ioc.rules.yaml` comme indicateur de cryptomining.

Limites observées

Durée d'observation : Une fenêtre de 15 secondes est suffisante pour détecter `fake-miner` en raison de son comportement immédiat et soutenu. Un malware sophistiqué pourrait implémenter un délai d'activation ou détecter l'environnement d'analyse et rester dormant, nécessitant des durées d'observation plus longues pour être détecté.

Réseau isolé et tentatives bloquées : Le réseau `--internal` bloque les connexions sortantes, empêchant le mineur de contacter un pool de minage réel. Les tentatives de connexion vers les ports 3333 et 4444 via `netcat` ne laissent pas de traces exploitables dans les logs du conteneur, réduisant la valeur de l'indicateur `ioc-persistent-connection` dans ce contexte.

Échantillon de test limité : Échantillon de test limité. Les résultats reposent sur deux images : une légitime et une malveillante simulée. Une évaluation plus rigoureuse nécessiterait un corpus plus large incluant des images aux comportements variés: images légitimes à forte consommation CPU, images malveillantes obfusquées à démarrage différé, pour mesurer les taux de faux positifs et de faux négatifs de manière statistiquement significative.

6.11.2 Résultats de l'analyse statique

Listing 50: Retour rapport nginx:latest

```
1 {
2   "analysis_type": "Static",
3   "image_name": "nginx:latest",
4   "xmrig_detected": false,
5   "kminer_detected": false,
6   "sh_detected": true,
7   "sh_samples": 8,
8   "curl_detected": true,
9   "curl_samples": 1,
10  "wget_detected": false,
11  "wget_samples": 0,
12  "higher_permission_detected": false,
13  "malicious_urls": [],
14  "malicious_files": [],
15  "malicious_pool": [],
16  "malicious_stratum": []
17 }
```

Listing 51: Retour CLI nginx:latest

```
1 =====
2 Demarrage de l'analyse statique de l'image
3 [*] Lecture du fichier de regles
4 [*] Lecture du fichier de regles termine
5 =====
6
7 [*] Extraction des indicateurs de compromission...
8 [*] Extraction termine
9 =====
10
11 [*] Analyse des couches de l'image de conteneur...
12 [*] Analyse des couches de l'images termine
13 =====
14
15 [*] Analyse du systeme de fichiers du conteneur.
16 [*] Creation du conteneur...
17 [*] Dressage de la liste des fichiers...
18 [*] Analyse du systeme de fichiers...
19 [*] Analyse du systeme de fichiers termine
20 =====
21
22 [*] Preparation des resultats...
23 [*] Nettoyage du conteneur.
24 Resultats de l'analyse statique:
25 =====
26   Presence xmrig: Non
27   Presence minerd: Non
28   Presence kminer: Non
29   Nombre de commandes "sh": 8
30   Nombre de commandes "curl": 1
31   Nombre de commandes "wget": 0
32   Liens url suspects: Aucun
33   Fichiers suspects dans le conteneur: Aucun
34   Liens pool suspects: Aucun
35   Liens stratum suspects: Aucun
```

36 =====

Listing 52: Retour rapport fake-miner:latest

```
1 {
2   "analysis_type": "Static",
3   "image_name": "fake-miner:latest",
4   "xmrig_detected": false,
5   "kminer_detected": true,
6   "sh_detected": true,
7   "sh_samples": 6,
8   "curl_detected": true,
9   "curl_samples": 2,
10  "wget_detected": false,
11  "wget_samples": 0,
12  "higher_permission_detected": false,
13  "malicious_urls": [
14    "stratum+tcp://pool.minexmr.com:4444'",
15    "http://pool.minexmr.com/api"
16  ],
17  "malicious_files": [
18    "/usr/bin/kworker"
19  ],
20  "malicious_pool": [
21    "stratum+tcp://pool.minexmr.com:4444'",
22    "http://pool.minexmr.com/api",
23    "stratum+tcp://pool.minexmr.com:4444'",
24    "POOL_URL=stratum+tcp://pool.minexmr.com:4444"
25  ],
26  "malicious_stratum": []
27 }
```

Listing 53: Retour CLI fake-miner:latest

```
1 =====
2 Demarrage de l'analyse statique de l'image
3 [*] Lecture du fichier de regles
4 [*] Lecture du fichier de regles termine
5 =====
6
7 [*] Extraction des indicateurs de compromission...
8 [*] Extraction termine
9 =====
10
11 [*] Analyse des couches de l'image de conteneur...
12 [*] Analyse des couches de l'images termine
13 =====
14
15 [*] Analyse du systeme de fichiers du conteneur.
16 [*] Creation du conteneur...
17 [*] Dressage de la liste des fichiers...
18 [*] Analyse du systeme de fichiers...
19 [*] Analyse du systeme de fichiers termine
20 =====
21
22 [*] Preparation des resultats...
23 [*] Nettoyage du conteneur.
24 Resultats de l'analyse statique:
25 =====
```

```

26  Presence xmrig: Non
27  Presence minerd: Non
28  Presence kminer: Oui
29  Nombre de commandes "sh": 6
30  Nombre de commandes "curl": 2
31  Nombre de commandes "wget": 0
32  Liens url suspects: ["stratum+tcp://pool.minexmr.com:4444'",
33  'http://pool.minexmr.com/api']
34  Fichiers suspects dans le conteneur: ['/usr/bin/kworker']
35  Liens pool suspects: ["stratum+tcp://pool.minexmr.com:4444'",
36  'http://pool.minexmr.com/api', "stratum+tcp://pool.minexmr.com:4444'",
37  'POOL_URL=stratum+tcp://pool.minexmr.com:4444']
38  Liens stratum suspects: Aucun
39  =====

```

Listing 54: Retour rapport 021982/xmrig:latest

```

1  {
2    "analysis_type": "Static",
3    "image_name": "021982/xmrig",
4    "xmrig_detected": true,
5    "kminer_detected": false,
6    "sh_detected": true,
7    "sh_samples": 10,
8    "curl_detected": false,
9    "curl_samples": 0,
10   "wget_detected": true,
11   "wget_samples": 2,
12   "higher_permission_detected": false,
13   "malicious_urls": [
14     "https://raw.githubusercontent.com/nhatquanglan/
15     daoovps/master/daoxmrig_ubuntu_tor_docker.py"
16   ],
17   "malicious_files": [
18     "/etc/dao.py"
19   ],
20   "malicious_pool": [],
21   "malicious_stratum": []
22 }

```

Listing 55: Retour CLI 021982/xmrig:latest

```

1  =====
2  Demarrage de l'analyse statique de l'image
3  [*] Lecture du fichier de regles
4  [*] Lecture du fichier de regles termine
5  =====
6
7  [*] Extraction des indicateurs de compromission...
8  [*] Extraction termine
9  =====
10
11  [*] Analyse des couches de l'image de conteneur...
12  [*] Analyse des couches de l'images termine
13  =====
14
15  [*] Analyse du systeme de fichiers du conteneur.
16  [*] Creation du conteneur...
17  [*] Dressage de la liste des fichiers...

```

```

18 [*] Analyse du systeme de fichiers...
19 [*] Analyse du systeme de fichiers termine
20 =====
21
22 [*] Preparation des resultats...
23 [*] Nettoyage du conteneur.
24 Resultats de l'analyse statique:
25 =====
26     Presence xmrig: Oui
27     Presence minerd: Non
28     Presence kminer: Non
29     Nombre de commandes "sh": 10
30     Nombre de commandes "curl": 0
31     Nombre de commandes "wget": 2
32     Liens url suspects: ['https://raw.githubusercontent.com/
33     nhathquanglan/daoyps/master/daoxmrig_ubuntu_tor_docker.py']
34     Fichiers suspects dans le conteneur: ['/etc/dao.py']
35     Liens pool suspects: Aucun
36     Liens stratum suspects: Aucun

```

Méthodologie d'évaluation

L'analyse statique est évaluée en le testant contre trois différentes images: l'image légitime nginx:latest, l'image malveillante créée fake-miner:latest et une image malveillante de la vie réelle 021982/xmrig. Les aspects les plus intéressants à comparer sont la détection des liens urls suspicieux pointant vers des binaires ou des scripts de cryptominage, des commandes de téléchargement telle que `curl`, des fichiers suspects générés lors de création du conteneur.

Nginx

L'image nginx est une image officielle offerte par Docker pour démarrer un serveur Web avec le cadriciel du même nom. Il était attendu que lors de l'analyse statique que cette image contiennent certainement des commandes `curl` ou `sh` pour mettre en place nginx, mais aussi qu'aucun lien url malicieux ou fichier suspect soit présent à l'intérieur. Finalement, cette hypothèse est confirmée, puisqu'aucun lien malicieux ni aucun fichier suspect ne sont apparus dans le rapport. Ainsi, l'outil ne génère pas de faux positifs à l'égard de nginx:latest.

Fake-miner

L'image fake-miner est une image qui a été créée principalement pour tester l'analyse dynamique sans compromettre la machine l'hébergeant, et cette image est aussi traitée par l'analyse statique. Le contenu de l'image étant déjà connu, contenant des faux liens vers des pools de cryptominage et un binaire obfusqué de cryptominage à l'aide de `kworker`. Après l'analyse statique, l'outil a détecté `kworker` et les liens pointant vers des pools et les a transcrits dans le rapport.

021982/xmrig

L'image 021982/xmrig est une vraie image malveillante étant utilisée pour mener à des attaques de crypto jacking sur la machine de la victime. Au préalable, une analyse manuelle a été menée pour comprendre le contenu de cette image. Cette analyse manuelle consistait simplement à consulter manuellement les couches de l'image pour lire les commandes suspectes. Lors de l'analyse statique de l'outil, celui-ci a détecté un script python provenant de github contenant `xmrig` dans son nom qui a été sauvegardé en autre nom `/etc/dao.py` dans le système de fichier d'un conteneur de l'image. Ce script pouvait contenir alors les commandes continuant l'installation de crypto jacking.

Limites Observées

Pas d'analyse des fichiers en profondeur : Le manque d'analyse des fichiers en profondeur peut mener à des résultats non conclusifs sur l'état de l'image. Dans l'exemple de l'image 021982/xmrig, l'outil n'analyse

pas `/etc/dao.py`, alors que ce fichier peut contenir encore plus d'informations pouvant confirmer la nature de l'image. Voir la sous section « Manque d'analyse en profondeur » des fichiers sous la section **Difficultés et limites rencontrées** pour plus d'informations.

Pas d'analyse de binaire : Le manque d'analyse de binaire soit par la comparaison des signatures des binaires et par la correspondance d'ensemble de bytes suspect permet à des binaires sophistiquement obfusqués d'échapper à l'analyse.

Petit échantillon de test : L'échantillon de test, comme mentionné dans la section d'analyse dynamique, n'est pas assez large pour être capable de détecter toutes les différentes formes de binaires, de liens url pointant vers des liens malicieux, et les liens vers des pools de crypto minage.

6.11.3 Réponse à la problématique

Les résultats obtenus permettent d'apporter des éléments de réponse à la problématique posée : dans quelle mesure une approche combinant l'analyse statique et l'analyse dynamique permet-elle d'identifier de manière plus fiable des images Docker impliquées dans des activités de cryptomining malveillant, tout en limitant les faux positifs?

Analyse dynamique

Sur la détection : L'analyse dynamique parvient à distinguer clairement les deux images testées. L'image légitime ne produit aucun IoC hit tandis que l'image malveillante en produit deux, avec des métriques CPU et mémoire significativement différentes. La combinaison de deux indicateurs indépendants:comportemental (CPU) et structurel (nom de binaire), qui renforce la fiabilité du verdict en réduisant la dépendance à un seul signal.

Sur la limitation des faux positifs : Un faux positif a été identifié et corrigé lors du développement : le mot `sh` dans les règles `ioc-url` déclenchait incorrectement une alerte sur `nginx` à cause des références à des scripts `.sh` dans ses logs d'initialisation. Ce problème a été résolu par un filtre contextuel vérifiant que `sh` apparaît comme commande et non comme extension. L'absence d'IoC hits sur `nginx` dans les résultats finaux confirme l'efficacité de ce correctif.

Sur la technique d'obfuscation : Le cas du binaire `kworker` illustre une limite importante de la détection par nom de binaire seule. Sans la règle `kworker` dans `ioc.rules.yaml`, seul l'IoC CPU aurait été déclenché. Ce résultat justifie l'approche combinée : l'analyse comportementale (CPU élevé soutenu) permet de détecter des malwares dont le nom de binaire est inconnu, tandis que la liste de binaires suspects permet d'identifier des malwares à faible consommation CPU ou à démarrage différé.

Analyse statique

Détection de comportements suspects : L'analyse statique est certainement capable de retrouver automatiquement des commandes suspectes dans les 2 images malveillantes `fake-miner` et `021982/xmrig`. À partir de ces commandes, l'analyse statique peut reconnaître des liens urls malveillants ou des liens vers des pools. Cependant, pour augmenter sa capacité de détection et rester à l'affût des nouvelles techniques d'attaque des, il faut ajouter plus d'indicateurs dans le fichiers `ioc.rules.yaml`.

Faux positifs : L'analyse statique n'a pas détecté des commandes suspectes ni des fichiers suspects dans l'image officielle `nginx`. Cependant, un léger problème avec l'analyse statique est qu'elle rapporte toute utilisation de la commande `sh` alors que celle-ci peut être tout à fait anodine. C'est le cas en effet lorsque l'analyse statique a rapporté son utilisation dans l'image `nginx`. Cependant, la présence de `sh` dans la commande n'est pas suffisante pour que l'analyse statique le relève comme une commande suspecte. L'analyse statique détermine la présence de d'autres indicateurs pour estimer la nature de la commande, causant moins de faux positifs.

6.12 Difficultés et limites rencontrées

6.12.1 Pas de support GPU réel

Docker ne permet pas d'accéder au GPU sans configuration spécifique (`--gpus` + drivers NVIDIA). La simulation GPU de `fake-miner` utilise du calcul CPU intensif, ce qui ne reflète pas exactement le comportement d'un vrai mineur GPU.

6.12.2 Réseau isolé et tentatives bloquées

Le réseau `--internal` bloque les connexions sortantes, ce qui empêcherait certains malwares de démarrer leur activité de minage. Les tentatives de connexion échouées ne laissent pas toujours de traces dans les logs du conteneur.

6.12.3 Durée d'observation fixe

L'outil observe le comportement pendant une durée fixe configurée par l'utilisateur. Un malware sophistiqué pourrait détecter l'environnement d'analyse et rester dormant pendant cette période.

6.12.4 Image de test personnalisée

On souhaitait tester avec une image infectée depuis l'internet pour s'assurer que les effets soient véridiques lors des tests. Nous avons cependant considéré les risques potentiels que pourrait engendrer l'utilisation d'une telle image et avons conclu que le coût n'en valait pas la peine ce qui à mener à faire une image infectée simulée selon nos indicateurs de compromission définis.

6.12.5 Manque d'analyse en profondeur des fichiers

Des scripts malveillants ou des binaires de crypto mineurs suffisamment obfusqués dans d'autres types de fichiers ou de scripts peuvent échapper à l'analyse statique. L'analyse en profondeur requiert beaucoup plus de temps et d'expertise que nous n'avons pas en ce moment, dont notamment sur l'analyse de fichiers en binaires. Avec plus de temps et de connaissance, il serait possible d'ajouter cette fonctionnalité à l'outil.

7 Conclusion

La conteneurisation est une technologie qui occupe une énorme place dans le développement de logiciel et le déploiement de systèmes. Cela a cependant mené à la popularisation des attaques à travers des images de crypto-jacking et ce problème n'est pas assez répondu par les outils populaires sur le marché d'analyse d'image. Cette tâche n'est pas forcément facile; les attaquants emploient des techniques sophistiquées de déploiement de crypto-jacking dans les images pour passer inaperçus. Ainsi, le projet voit cette importance d'un outil qui doit répondre aux attaques de crypto-jacking à travers les conteneurs de l'image. En choisissant de mener à la fois une analyse statique et une analyse dynamique sur les images, l'outil pourra détecter des comportements suspects dans l'image et déterminer si celle-ci contient du crypto-jacking. En effet, l'analyse dynamique évalue si un conteneur de l'image dépasse une certaine quantité de ressources des processeurs (CPU et GPU) et d'utilisation de la carte réseau de l'ordinateur. Quant à l'analyse statique, celle-ci cherche à traverser les couches de la création de l'image pour trouver des commandes semblant d'installer du crypto-jacking et des fichiers suspects dans le conteneur de l'image. De plus, cet outil peut s'intégrer dans un pipeline CI/CD pour détecter des images malveillantes lorsque celles-ci sont utilisées dans le cycle de test et de déploiement.

Après l'implémentation de l'outil, il est capable de détecter du comportement suspect dans les images malveillantes de conteneur. En particulier, l'analyse dynamique semble la plus performante des deux dans l'état actuel, même si l'échantillon de test reste petit. Cette analyse est capable de détecter les processus qui consomment le plus de ressources d'ordinateur et d'attribuer un caractère malveillant à ceux-ci. Quant à l'analyse statique, celle-ci est certes capable de relever des commandes suspectes et des fichiers suspects créés par ces commandes, mais elle est limitée par le manque d'indicateurs et le manque d'analyse en profondeur des fichiers suspects. Malgré cela, notre outil a le potentiel de répondre à la problématique.

Pour accomplir ce potentiel, l'outil doit ajouter de nouvelles fonctionnalités. Notamment, l'analyse statique pourrait implémenter l'analyse des fichiers suspects, qu'ils soient des scripts d'installation ou des binaires. L'analyse de binaires requiert plus d'expertise dans le domaine, mais cette tâche pourrait être déléguée à un autre outil sur le marché.

Sur le plan de l'analyse dynamique, les résultats obtenus démontrent que l'approche comportementale est efficace pour identifier des images malveillantes simulées. L'outil est capable de détecter une utilisation soutenue du CPU dépassant le seuil configuré, d'identifier des processus aux noms suspects et de capturer les tentatives de connexion réseau vers des ports associés au minage. Cependant, l'utilisation d'images simulées constitue la principale limite de cette évaluation. Ces images reproduisent les comportements observables du cryptomining (consommation CPU élevée, processus obfusqués, tentatives de connexion réseau) mais ne reflètent pas toute la sophistication des malwares réels, qui peuvent implémenter des délais d'activation, détecter les environnements d'analyse ou moduler leur consommation de ressources pour passer sous les seuils de détection. Pour pallier cette limite, des tests sur des images malveillantes réelles documentées dans la littérature académique seraient nécessaires, mais cela requiert une infrastructure de sécurité renforcée avec des mécanismes de confinement plus robustes que ceux mis en place dans ce prototype. Enfin, l'intégration d'un système de scoring combinant les résultats statiques et dynamiques, actuellement absent de l'implémentation, permettrait de réduire les faux positifs et d'offrir un verdict plus nuancé qu'une simple liste d'indicateurs détectés.

References

- [1] BleepingComputer. *Docker Hub Repositories Hide over 1,650 Malicious Containers*. 2026. URL: <https://www.bleepingcomputer.com/news/security/docker-hub-repositories-hide-over-1-650-malicious-containers/> (urlseen 17/02/2026).
- [2] Jacopo Bufalino **and others**. ?ORCA: Unveiling Obscure Containers In The Wild? *in arXiv preprint arXiv:2509.09322*: (11 **september** 2025). DOI: [10.48550/arXiv.2509.09322](https://doi.org/10.48550/arXiv.2509.09322). URL: <https://doi.org/10.48550/arXiv.2509.09322> (urlseen 12/02/2026).
- [3] Check Point Software. *XM Rig Malware*. 2026. URL: <https://www.checkpoint.com/cyber-hub/threat-prevention/what-is-malware/xmrig-malware/> (urlseen 03/03/2026).
- [4] *Crypto Mining Tools for 2026 Beginner*. 8 **may** 2025. URL: <https://www.cryptominerbros.com/blog/crypto-mining-tools-for-beginner/>.
- [5] Markus Dahlmanns **and others**. ?Secrets Revealed in Container Images: An Internet-Wide Study on Occurrence and Impact? *in arXiv preprint arXiv:2307.03958*: (2023). DOI: [10.48550/arXiv.2307.03958](https://doi.org/10.48550/arXiv.2307.03958). URL: <https://doi.org/10.48550/arXiv.2307.03958> (urlseen 12/02/2026).
- [6] *Docker Hub Images Downloaded 20M Times Come with Cryptominers*. 2026. URL: <https://www.bleepingcomputer.com/news/security/docker-hub-images-downloaded-20m-times-come-with-cryptominers/> (urlseen 17/02/2026).
- [7] *Docker Scout Image Analysis*. 21 **february** 2026. URL: <https://docs.docker.com/scout/explore/analysis/>.
- [8] *ENISA Threat Landscape 2021*. 6 **november** 2025. URL: <https://www.enisa.europa.eu/publications/enisa-threat-landscape-2021> (urlseen 13/02/2026).
- [9] Freeman Law. *Mining Explained: A Detailed Guide on How Cryptocurrency Mining Works*. 2026. URL: <https://freemanlaw.com/mining-explained-a-detailed-guide-on-how-cryptocurrency-mining-works/> (urlseen 16/02/2026).
- [10] Mubin Ul Haque **and** M. Ali Babar. ?Well Begun Is Half Done: An Empirical Study of Exploitability & Impact of Base-Image Vulnerabilities? *in arXiv preprint arXiv:2112.12597*: (21 **december** 2021). DOI: [10.48550/arXiv.2112.12597](https://doi.org/10.48550/arXiv.2112.12597). URL: <https://doi.org/10.48550/arXiv.2112.12597>.
- [11] *Kaspersky Report on Cryptojacking in Q1–Q3 2022*. 10 **november** 2022. URL: <https://securelist.com/cryptojacking-report-2022/107898/> (urlseen 13/02/2026).
- [12] R. Kim **and others**. ?Detecting Cryptojacking Containers Using eBPF-Based Security Runtime and Machine Learning? *in Electronics*: 14 (2025), **page** 1208. DOI: [10.3390/electronics14061208](https://doi.org/10.3390/electronics14061208). URL: <https://doi.org/10.3390/electronics14061208> (urlseen 12/02/2026).
- [13] MITRE ATT&CK. *User Execution: Malicious Image (T1204.003)*. 2026. URL: <https://attack.mitre.org/techniques/T1204/003/> (urlseen 07/02/2026).
- [14] Alexander Nelson **and others**. *Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile*. NIST Special Publication 800-61 Rev. 3. National Institute of Standards **and** Technology, 3 **april** 2025. DOI: [10.6028/NIST.SP.800-61r3](https://doi.org/10.6028/NIST.SP.800-61r3). URL: <https://doi.org/10.6028/NIST.SP.800-61r3> (urlseen 12/02/2026).
- [15] Murugiah Souppaya **and** Karen Scarfone. *Guide to Malware Incident Prevention and Handling for Desktops and Laptops*. NIST Special Publication 800-83 Rev. 1. National Institute of Standards **and** Technology, 22 **july** 2013. DOI: [10.6028/NIST.SP.800-83r1](https://doi.org/10.6028/NIST.SP.800-83r1). URL: <https://doi.org/10.6028/NIST.SP.800-83r1> (urlseen 12/02/2026).
- [16] Sysdig. *Analysis on Docker Hub Malicious Images: Attacks through Public Container Images*. 2026. URL: <https://www.sysdig.com/blog/analysis-of-supply-chain-attacks-through-public-docker-images> (urlseen 17/02/2026).
- [17] The Hacker News. *New Cryptojacking Attack Targets Docker API to Create Malicious Swarm Botnet*. 2024. URL: <http://thehackernews.com/2024/10/new-cryptojacking-attack-targets-docker.html> (urlseen 13/02/2026).

- [18] *Trivy - Container Image*. 2026. URL: https://trivy.dev/v0.69/guide/target/container_image/ (**urlseen** 10/02/2026).
- [19] Uptycs. *Team TNT Deploys Malicious Docker Image on Docker Hub*. 2026. URL: <https://www.uptycs.com/blog/threat-research-report-team/team-tnt-deploys-malicious-docker-image-on-docker-hub-with-pentesting-tools> (**urlseen** 08/02/2026).
- [20] Katrine Wist **and others**. ?Vulnerability Analysis of 2500 Docker Hub Images? *in arXiv preprint arXiv:2006.02932*: (11 **june** 2020). DOI: [10.48550/arXiv.2006.02932](https://doi.org/10.48550/arXiv.2006.02932). URL: <https://doi.org/10.48550/arXiv.2006.02932>.
- [21] Anastasiia Yevdokimova. *XM Rig Coin Miner: Adversaries Employ New Approaches to Illegal Crypto Mining*. 13 **july** 2022. URL: <https://socprime.com/blog/xmrig-coin-miner-adversaries-employ-new-approaches-in-illegal-crypto-mining/>.

A Contribution

A.1 Contributions des membres de l'équipe

Pour chaque tâche, le responsable devait:

- réaliser l'implémentation technique
- produire la documentation associée
- valider le bon fonctionnement

Membre	Tâches effectuées
(LCH) Côté-Handfield, Léna	• Rédaction de la section Problématique à résoudre • Rédaction de la section Définition de l'implémentation • Rédaction de la section Plan de projet • Implémentation du daemon.json • Implémentation du module main.py (commandes d'utilisation de l'outil) • Implémentation du module dynamic.py • Implémentation du module parser.py • Implémentation des fichiers de test (docker-compose.yml, .gitlab-ci.yml, fake-miner) • Rédaction de l'intégration dans un pipeline CI/CD • Rédaction de la section Résultats: Analyse Dynamique • Rédaction de la section Conclusion • Rédaction de la section Difficultés et limites rencontrées
(AH) Halimi, Amine	• Rédaction de la section État de l'art: Outils existants. • Implémentation du module static.py. • Rédaction de la section Problématique à résoudre. • Rédaction de la section Résultats: Analyse Statique. • Rédaction de la section Conclusion. • Implémentation du script cryptojack-detector.sh • Rédaction de la section Difficultés et limites rencontrées
(AH) Hébert, Alphée	• Rédaction des sections État de l'art: Cas de Uptycs, État de l'art: Indicateurs utiles, État de l'art: Contexte et objectifs • Rédaction des diapositives de la présentation. • Implémentation des modules report.py et interface.py. • Rédaction de la section Difficultés et limites rencontrées

Table 5: Attribution des tâches par membre